

JOI 2011/2012 本選

01:

JJO0II

解説： 保坂 和宏

問題概要

- J, O, I からなる文字列 S が与えられたとき, S に含まれる JOI 列のレベルの最大値を求める

「レベル 0 の JOI 列」… (空文字列)

「レベル 1 の JOI 列」… JOI

「レベル 2 の JOI 列」… JJOOII

「レベル 3 の JOI 列」… JJJOOOIII

.....

問題概要

- 例 : $S = \text{OJJOOIIOJOI}$
 - レベル 0 の JOI 列は含まれる
 - レベル 1 の JOI 列は含まれる (OJJOOIIOJOI)
 - レベル 2 の JOI 列は含まれる (OJJOOIIOJOI)
 - レベル 3 以上の JOI 列は含まれない
- JOI 列はつながって現れるものを考える
 - JOIJOIJOIJOIJOI にはレベル 2 の JOI 列は含まれない

問題概要

- 制約：文字列 S の長さ N は, $1 \leq N \leq 10^6$
 - 部分点 (20%) : $1 \leq N \leq 100$

単純な解法

- 各 k に対し, レベル k の JOI 列が S に含まれるかどうかを調べる
 - $0 \leq k \leq N/3$ を考えればよい
 - S の中でどこから始まるかをすべて考える
 - $3k$ 文字がレベル k の JOI 列であるか調べる
 - 時間計算量 $O(N^3)$

やや効率の良い解法

- 各 k に対し, レベル k の JOI 列が S に含まれるかどうかを調べる
 - $0 \leq k \leq N/3$ を考えればよい
 - S の中でどこから始まるかをすべて考える
 - $3k$ 文字がレベル k の JOI 列であるか調べる
 - 時間計算量 $O(N^3) \rightarrow O(N^2)$
- レベル k の JOI 列であるかの判定を高速にしたい

やや効率の良い解法

- 「 S の $i + 1$ 文字目から $i + k$ 文字目までがすべて文字 x であるか」がわかればよい
 - $(i + 1$ 文字目から $i + k$ 文字目までの x の個数)
= $(i + k$ 文字目までの x の個数)
– $(i$ 文字目までの x の個数)
 - 「 S の j 文字目までの文字 x の個数」を各 j
($0 \leq j \leq N$) について予め計算しておけばよい
 - 前処理 $O(N)$ / 判定 $O(1)$

やや効率の良い解法

J J O J I O J J J O O I I
0 1 2 2 3 3 3 4 5 6 6 6 6 6

J J O J I O J J J O O I I
0 1 2 2 3 3 3 4 5 6 6 6 6 6

J J O J I O J J J O O I I
0 1 2 2 3 3 3 4 5 6 6 6 6 6

効率の良い解法 (1)

- レベル k の JOI 列 ($k \geq 1$) の特徴的な部分
 - 文字 O が k 個連続している
 - その左右には O でない文字が来ている

JJJJOOOOIIII

効率の良い解法 (1)

- S に「文字 O が k 個連続していて、その左右には O でない文字が来ている」というような箇所があれば、その部分を使う JOI 列のレベルは k

✕ ...JOOOOI...

JJJOOOIII

✕ JJJJOOOOIIII

JJJJJOOOOOIIII

効率の良い解法 (1)

- S の「文字 O が連続していて、その左右には O でない文字が来ている」というような箇所をすべて求める
 - $O(N)$
- それぞれの箇所に対し、文字 O の個数を k としたとき、左側 k 文字がすべて J であるか・右側 k 文字がすべて I であるかを判定する
 - 先程の方法を用いれば、各箇所について $O(1)$

効率の良い解法 (1')

- それぞれの箇所に対し、文字 0 の個数を k としたとき、左側 k 文字がすべて J であるか・右側 k 文字がすべて I であるかを判定する
 - 実は工夫は不要で、左側 k 文字・右側 k 文字をすべて見てもよい
 - (見ることになる文字数) = ($2k$ の和)
= $2 \times (S \text{ に現れる文字 } 0 \text{ の個数}) \leq 2N$

効率の良い解法 (2)

- JOI 列を探しやすくなるように S を加工
- Run-Length Encoding
 - 文字列の圧縮に用いられる基本的な手法の 1 つ
- 同じ文字が連続する箇所をその長さで表す

OOJJJJJJJJOOOOO|||||



[O × 2] [J × 7] [O × 4] [| × 5]

効率の良い解法 (2)

- Run-Length Encoding を行った結果に対して, JOI 列は $[J \times a] [O \times b] [I \times c]$ という形の箇所にしか現れない
 - $a \geq b$ かつ $b \leq c$ のとき, レベル b の JOI 列が現れる
 - そうでないとき, JOI 列は現れない
 - 例 :
 - $(a, b, c) = (7, 4, 5)$: JJJJJJJOOOOIII
 - $(a, b, c) = (1, 2, 3)$: JOOIII

効率の良い解法 (2)

- Run-Length Encoding を行う
 - 時間計算量・空間計算量 $O(N)$
- 連続する 3 パートごとに,
[J × a] [O × b] [I × c] ($a \geq b$ かつ $b \leq c$) という形かを確認する
 - $O(N)$

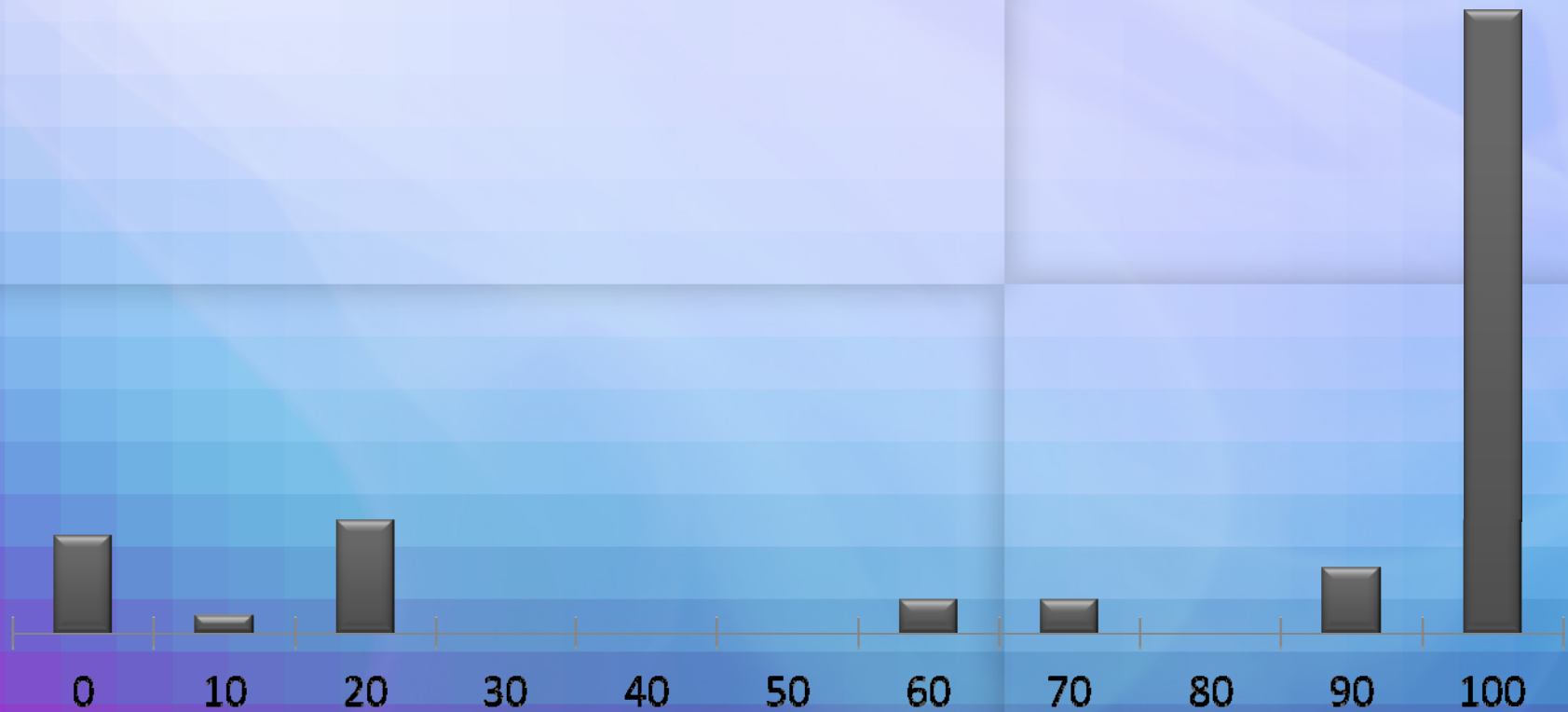
効率の良い解法 (2')

- 簡単に言い換えたアルゴリズム (メモリも節約)
- 以下を繰り返す：
 - S の先頭から J, O, I 以外の文字を取れるだけ取る (今回は不要)
 - S の先頭から文字 J を取れるだけ取る
 - S の先頭から文字 O を取れるだけ取る
 - S の先頭から文字 I を取れるだけ取る
 - J を a 個, O を b 個, I を c 個取れたとして, $a \geq b$ かつ $b \leq c$ ならばレベル b の JOI 列あり

まとめ

- JOI 列を効率良く探す
- JOI 列をどこから探せばよいかを考える
- Run-Length Encoding
- その他の解法もいろいろ

結果



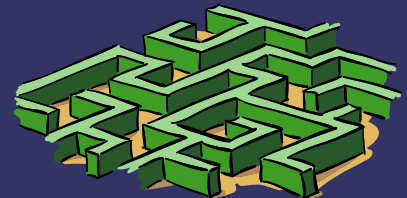
Card Game is Fun
本選第二問 たのしいカードゲーム

解説: 平野湧一郎



問題概要

- 1から1000までの数字が書かれたカードの山をアンナとブルーノが持っている
- アンナは任意のカードを取って新しい山を作る
- ブルーノは真ん中らへんを取ってきて新しい山を作る
- アンナとブルーノの山が一致するような最大の枚数を求めたい
- 最初の山の枚数は最大5000



例

- Anna: 1 2 3 4 5
- Bruno: 3 1 4 1



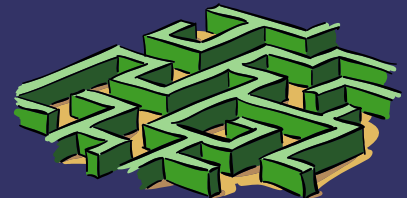
例

- Anna: 1 2 3 4 5
- Bruno: 3 1 4 1
- 二人は得点2を得る



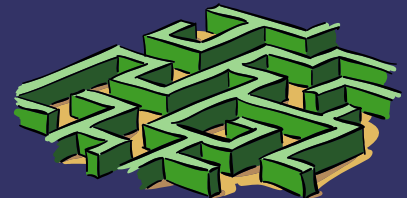
解き方1(カづく)

- ⤵ アンナが作れる全ての山について、ブルーノが作れるかどうか調べる
- ⤵ 最大のものが答え



解き方1の問題点

- ⇒ 例えばアンナの山が「1 2 3」だったら、調べる対象は8通り
 - 1 2 3
 - 1 2
 - 2 3
 - 1 3
 - 1
 - 2
 - 3
 - (なし)
- ⇒ 一般に、調べる対象は 2^A 通り
- ⇒ $A=10$ ぐらいまでならなんとかできるが、それ以上だと時間内に終わらない



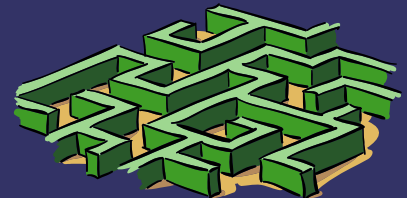
解き方2 (*Bruno-first*)

- ⇒ 逆に、ブルーノが作れる山それぞれについて、アンナが作れるかどうか調べる
- ⇒ ブルーノは山の連続した部分しか取れないため、調べる数は高々 B^2 通り
- ⇒ $O(AB^2)$
- ⇒ $A \leq 100, B \leq 100$ ならOK



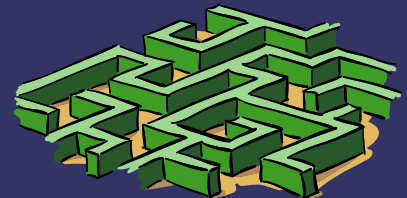
解き方2の問題点

- ⤵ 解き方2では、ブルーノの山の上から何枚捨てるか・下から何枚捨てるかを固定して考えている
 - ⤵ 下から捨てる枚数を固定する必要があるの？
 - 例えば $B=10$ の場合、
 - 山の3～8枚目を持ってきて新しい山にする
(下から2枚捨てる)
 - 山の3～9枚目を持ってきて新しい山にする
(下から1枚捨てる)
- という二つの処理は、山の3～8枚目まではまったく同じ探索を行っている
- ⤵ 上から何枚捨てるかだけを固定し、下から捨てる枚数はできるだけ少なくなるようにすればよい



解き方3(想定解)

- ⇒ ブルーノの山の上から $m-1$ 枚捨て、 m 枚目から連続で何枚取れるかを調べる
- ⇒ 上記の操作を $m=1$ から $m=B$ まで行い、最大値が答え
- ⇒ $O(AB)$



例

- Anna: 1 2 3 4 5
- Bruno: 3 1 4 5 2



例

- Anna: 1 2 **3** 4 5
- Bruno: **3** 1 4 5 2



例

- Anna: 1 2 3 4 5
- Bruno: 3 1 4 5 2



例

- Anna: 1 2 3 **4** **5**
- Bruno: 3 1 **4** **5** 2



例

- Anna: 1 2 3 4 **5**
- Bruno: 3 1 4 **5** 2



例

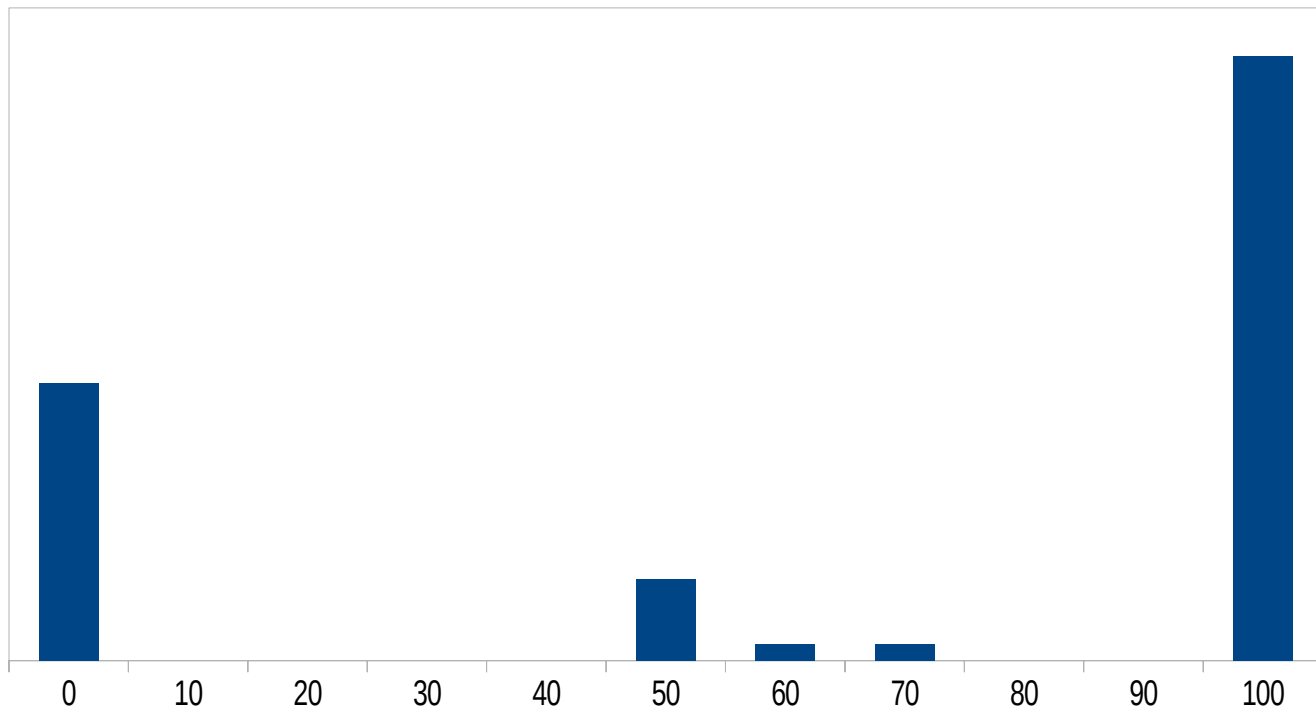
⇒ Anna: 1 2 3 4 5

⇒ Bruno: 3 1 4 5 2

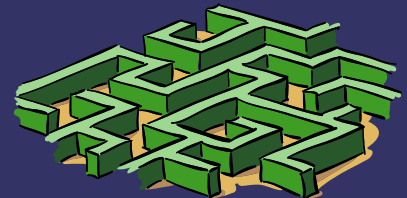
⇒ 答え = 3



結果



- 平均点: 66.9点
- 標準偏差: 44.0点



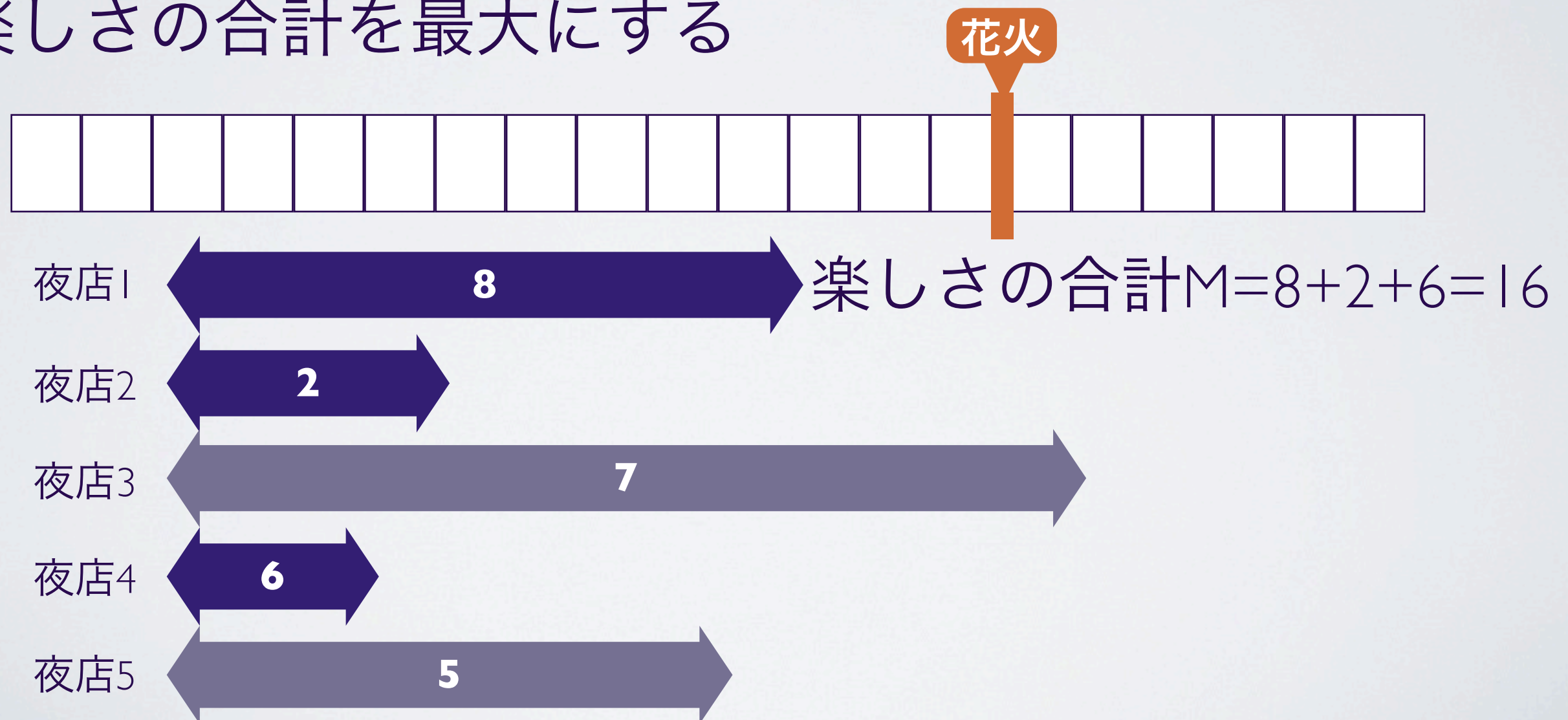


問題3 夜店(Night Market)

山下 洋史

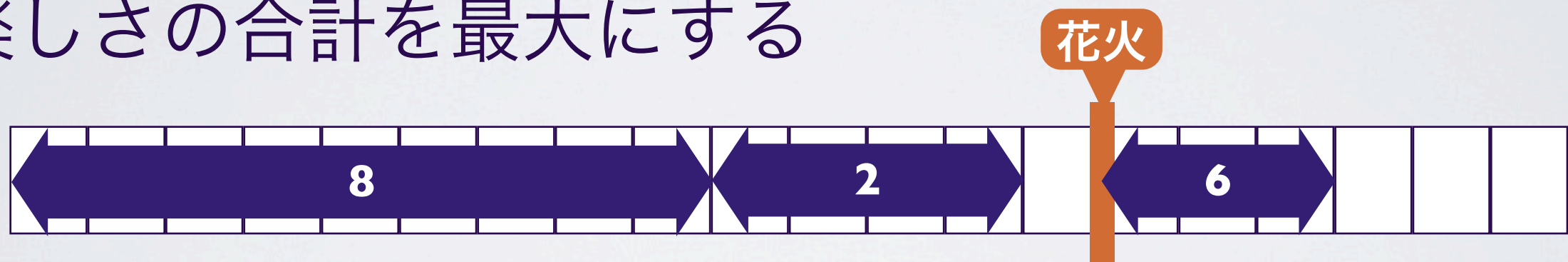
問題のおさらい

- N個の夜店のなかから順番通りにいくつか選んで遊ぶ
- 夜店で遊べる時間はTだけ
- 遊んでいる時間が時刻Sに被ってはいけない
- 楽しさの合計を最大にする



問題のおさらい

- N個の夜店のなかから順番通りにいくつか選んで遊ぶ
- 夜店で遊べる時間はTだけ
- 遊んでいる時間が時刻Sに被ってはいけない
- 楽しさの合計を最大にする



夜店1

楽しさの合計 $M = 8 + 2 + 6 = 16$

夜店2

夜店3

7

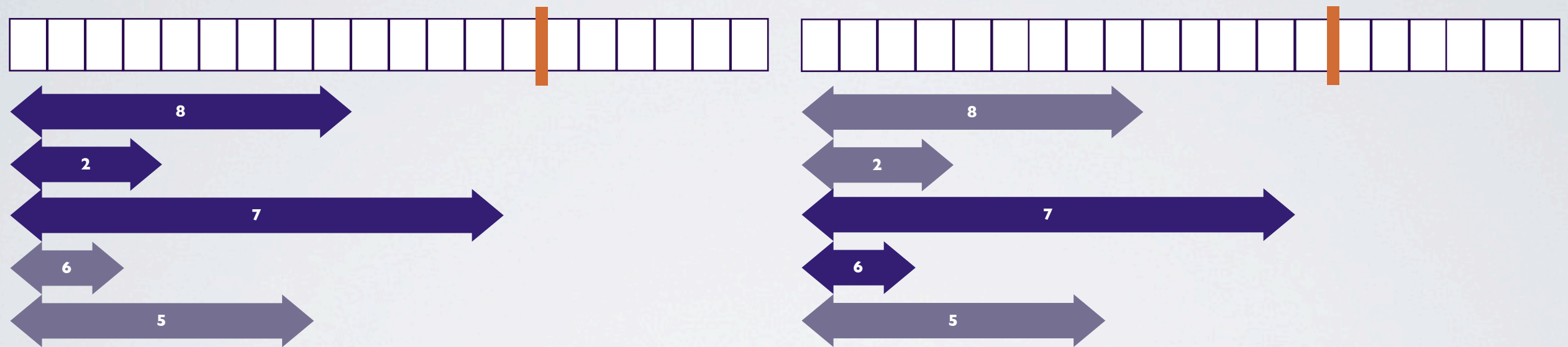
夜店4

夜店5

5

まずは力技で

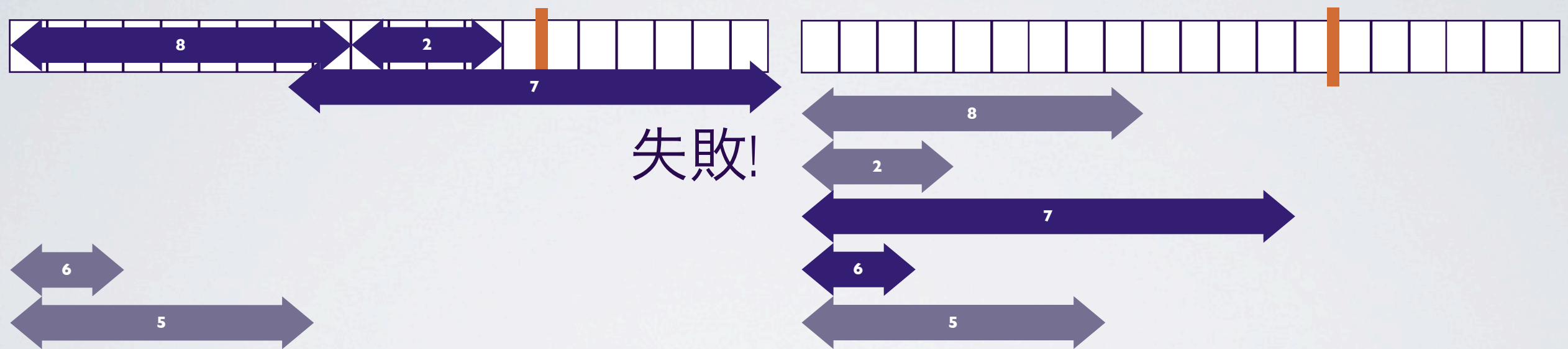
- とりあえず全通り選んでみて入れてみる



- $N=20$ なら $2^{20} = \text{約} 1000000 (10^6)$ 通り試せばできる

まずは力技で

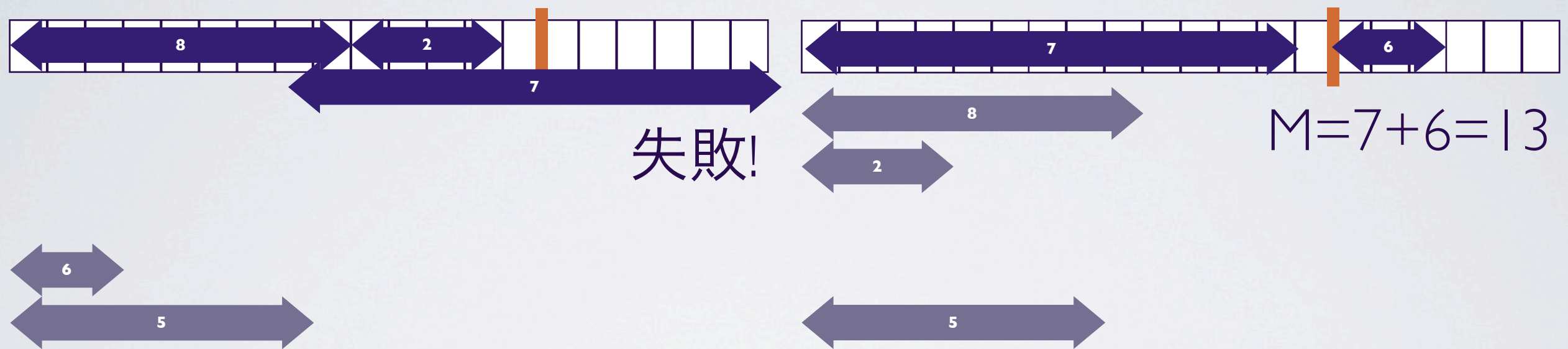
- とりあえず全通り選んでみて入れてみる



- $N=20$ なら $2^{20} = \text{約} 10000000 (10^6)$ 通り試せばできる

まずは力技で

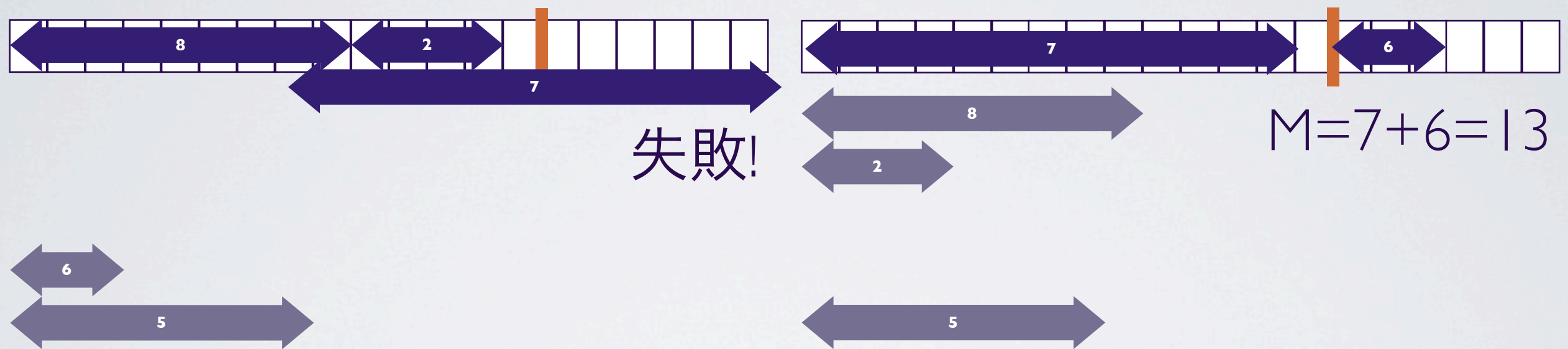
- とりあえず全通り選んでみて入れてみる



- $N=20$ なら $2^{20} = \text{約} 1000000 (10^6)$ 通り試せばできる

まずは力技で

- とりあえず全通り選んでみて入れてみる

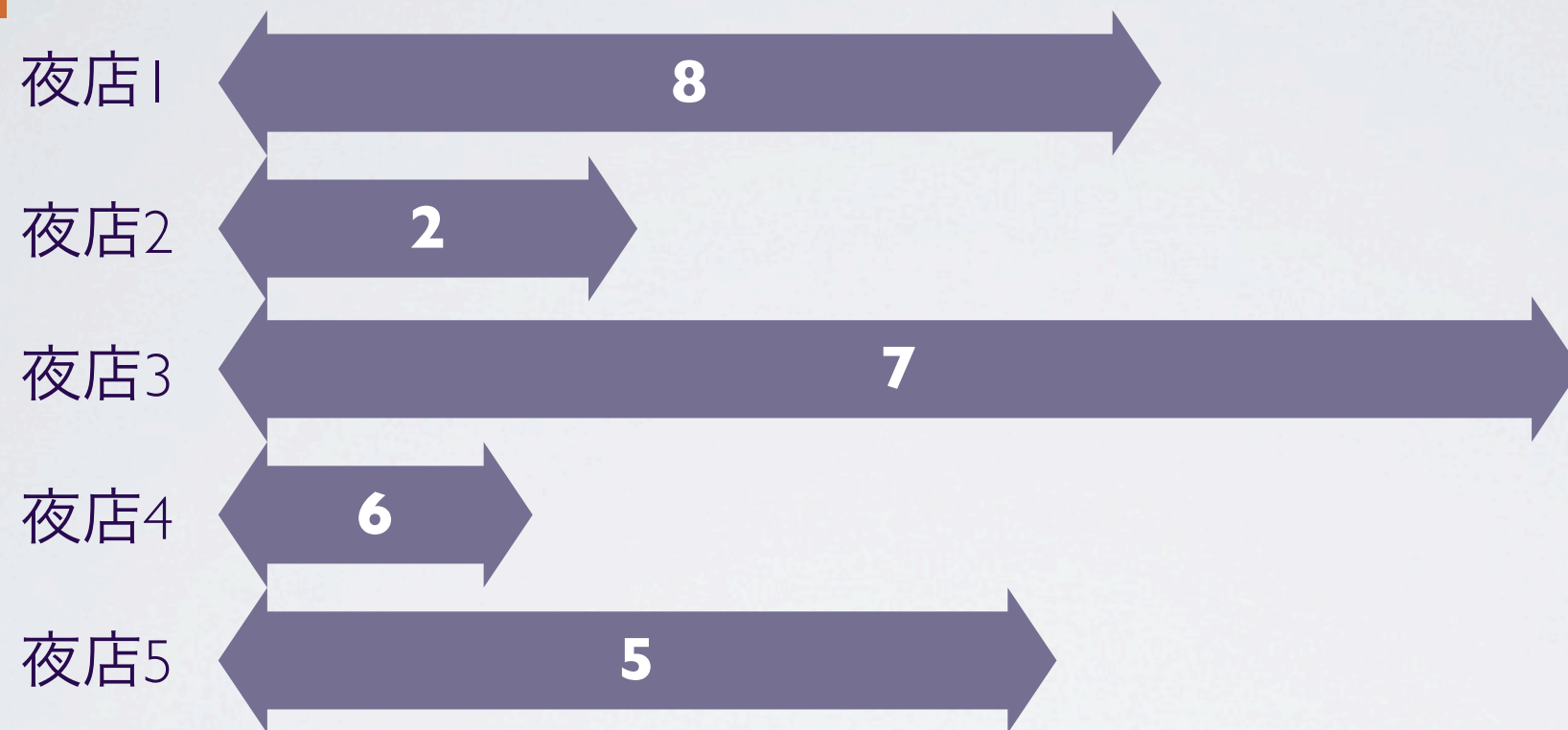


- $N=20$ なら $2^{20} = \text{約} 1000000 (10^6)$ 通り試せばできる
- $N=3000$ なら $2^{3000} = 100 \dots 00 (10^{900} \text{ぐらい})$ も！
- 10%以上の得点は無理そう...
- $O(2^N)$

とりあえず10点

もしも花火が到着の瞬間に打ち上がったなら

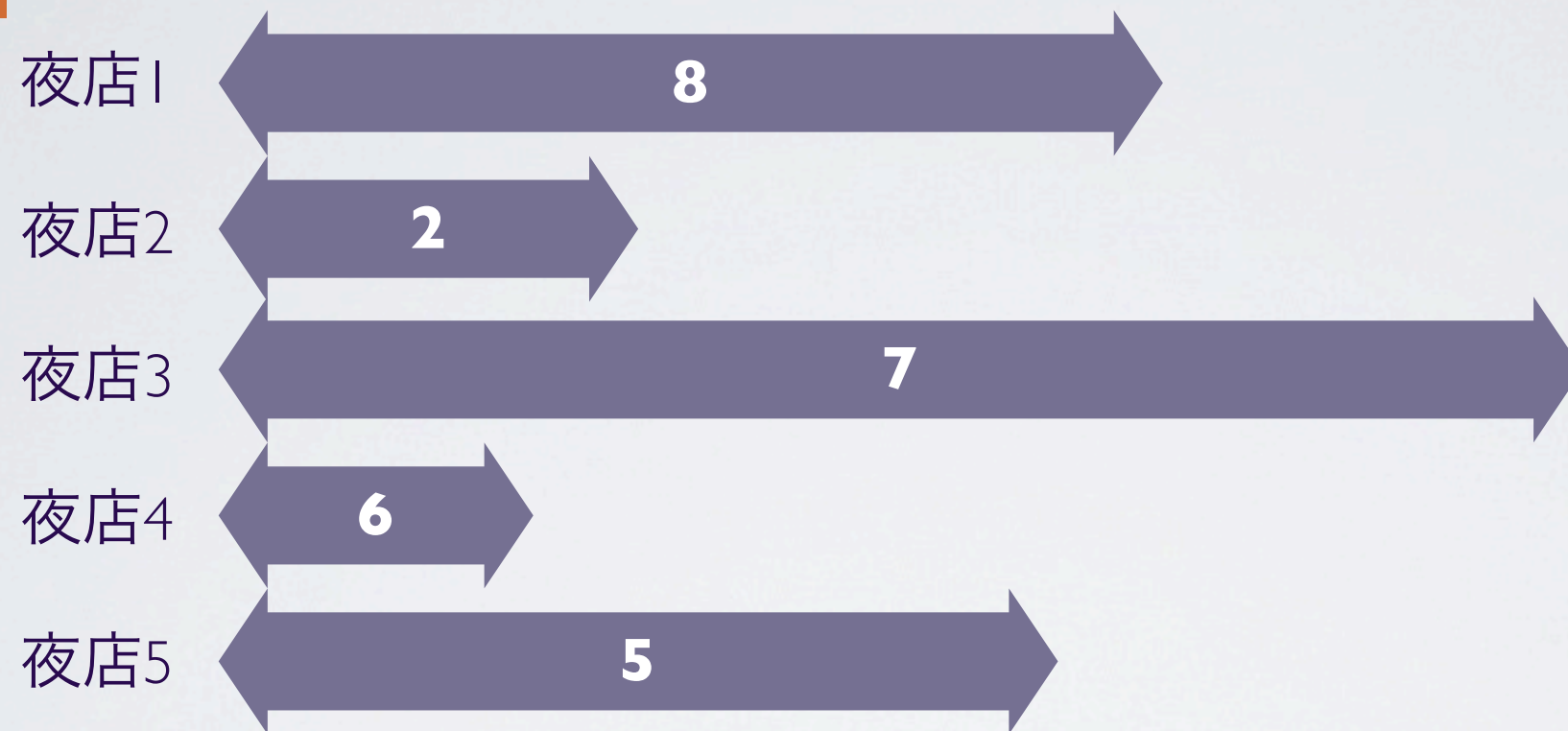
花火



- 花火大会がなくなったのと同じ状況
- ちょっと簡単になった. でもどうしよう？

もしも花火が到着の瞬間に打ち上がったなら

花火



- 花火大会がなくなったのと同じ状況
 - ちょっと簡単になった. でもどうしよう?
- 動的計画法(DP)が使える

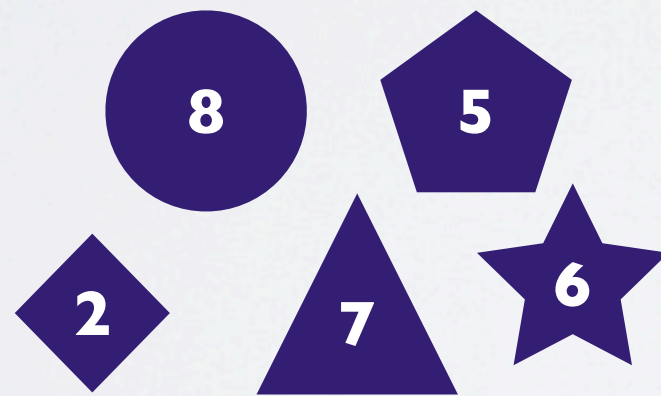
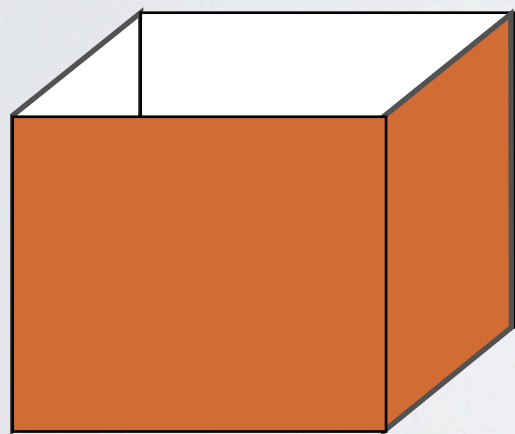
動的計画法(DP)って何さ

- おなじみの人にはおなじみDynamic Programming
- ある最適化問題を複数の部分問題に分割して解く際に、そこまでに求められている以上の最適解が求められないような部分問題を切り捨てながら解いていく手法である。
(Wikipediaより)
- 100番目のフィボナッチ数(1,1,2,3,5,8,...)の計算で例える
 - k番目までのフィボナッチ数が分かっていた（部分問題）ら、k-1番目の物とk番目の物を足せばk+1番目のものが分かる
 - これを繰り返すと100番目のフィボナッチ数がわかる
 - 途中まで計算した結果を用いて次の計算をする

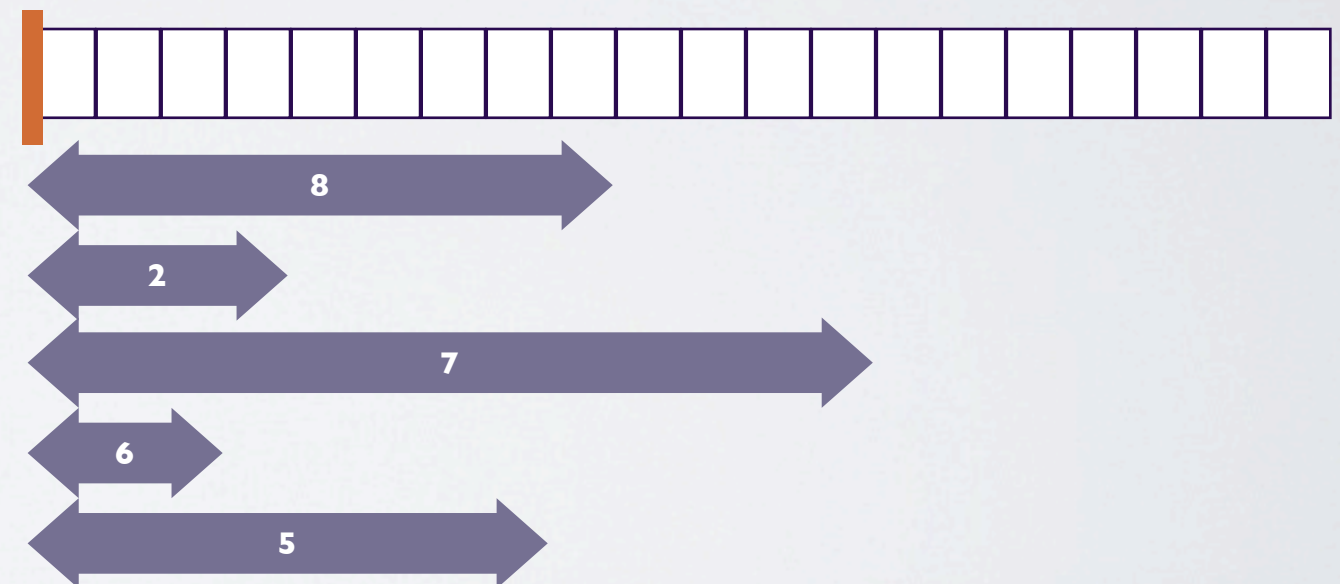
ナップサック問題

- 動的計画法(DP)の典型的な応用例
- ナップサックに, いろんな体積と値段の荷物を入れる
- 荷物の体積の合計をナップサックの体積以下にしながら, 値段の合計を出来るだけ大きくする

ナップサック問題



今回の問題($S=0$ の場合)



一緒！

やってみよう

- 時刻 t までに、夜店 i までを遊ぶときの楽しさの最高をもとめる(その答えを $dp(t,i)$ と置いておきます)
- $dp(t,i+1)$ を求めたい
 - 夜店 $i+1$ で遊ぶ
 - 時刻 $(t - B_{i+1})$ で夜店 i まで遊んだ時の楽しさ $+ A_{i+1}$
 - 夜店 $i+1$ で遊ばない
 - 時刻 t で夜店 i まで遊んだ時の楽しさ
- この二つの大きい方を取る！

表を埋める

- $dp(t, i+1) = \text{下の2つのmax}$
 - 夜店 $i+1$ で遊ぶ $\rightarrow$ 時刻 $(t - B_{i+1})$ で夜店 i まで遊んだ時の楽しさ $+ A_{i+1}$
 - 夜店 $i+1$ で遊ばない $\rightarrow$ 時刻 t で夜店 i まで遊んだ時の楽しさ
- 数式で書くと $dp(t, i+1) = \max\{dp(t - B_{i+1}, i) + A_{i+1}, dp(t, i)\}$

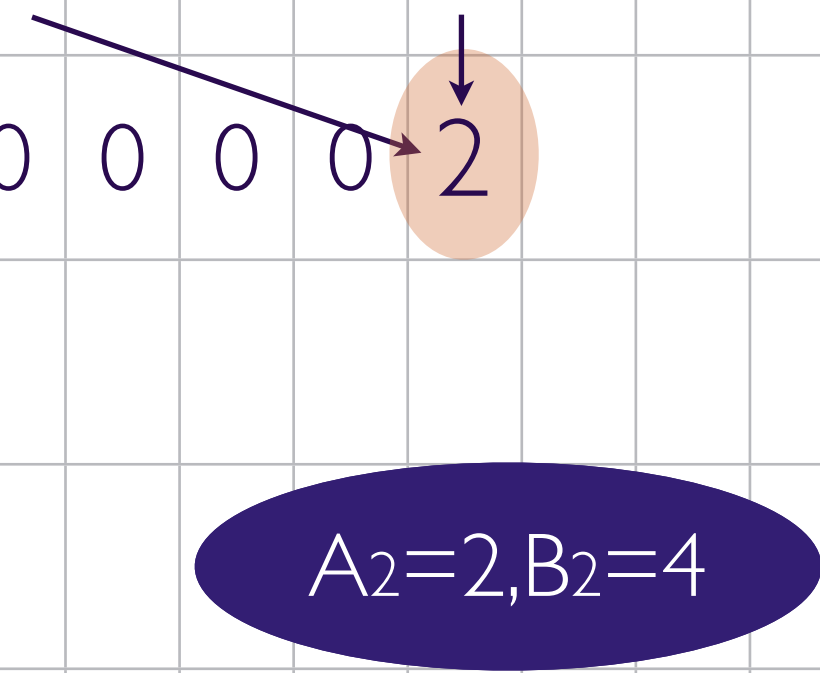
dp	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
1	0	0	0	0	0	0	0	0	0	8	8	8	8	8	8	8	8	8	8	8	8
2	0	0	0	0																	
3																					
4																					
5																					

$A_2=2, B_2=4$

表を埋める

- $dp(t, i+1) = \text{下の2つのmax}$
 - 夜店 $i+1$ で遊ぶ $\rightarrow$ 時刻 $(t - B_{i+1})$ で夜店 i まで遊んだ時の楽しさ $+ A_{i+1}$
 - 夜店 $i+1$ で遊ばない $\rightarrow$ 時刻 t で夜店 i まで遊んだ時の楽しさ
- 数式で書くと $dp(t, i+1) = \max\{dp(t - B_{i+1}, i) + A_{i+1}, dp(t, i)\}$

dp	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
1	0	0	0	0	0	0	0	0	0	8	8	8	8	8	8	8	8	8	8	8	8
2	0	0	0	0	2																
3																					
4																					
5																					

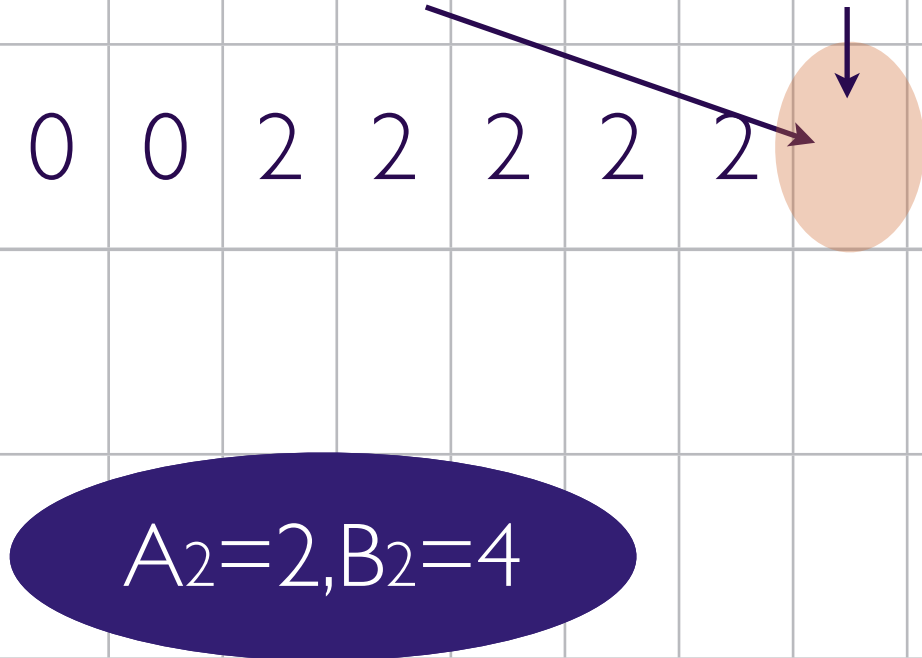


$A_2=2, B_2=4$

表を埋める

- $dp(t, i+1) = \text{下の2つのmax}$
 - 夜店 $i+1$ で遊ぶ $\rightarrow$ 時刻 $(t - B_{i+1})$ で夜店 i まで遊んだ時の楽しさ $+ A_{i+1}$
 - 夜店 $i+1$ で遊ばない $\rightarrow$ 時刻 t で夜店 i まで遊んだ時の楽しさ
- 数式で書くと $dp(t, i+1) = \max\{dp(t - B_{i+1}, i) + A_{i+1}, dp(t, i)\}$

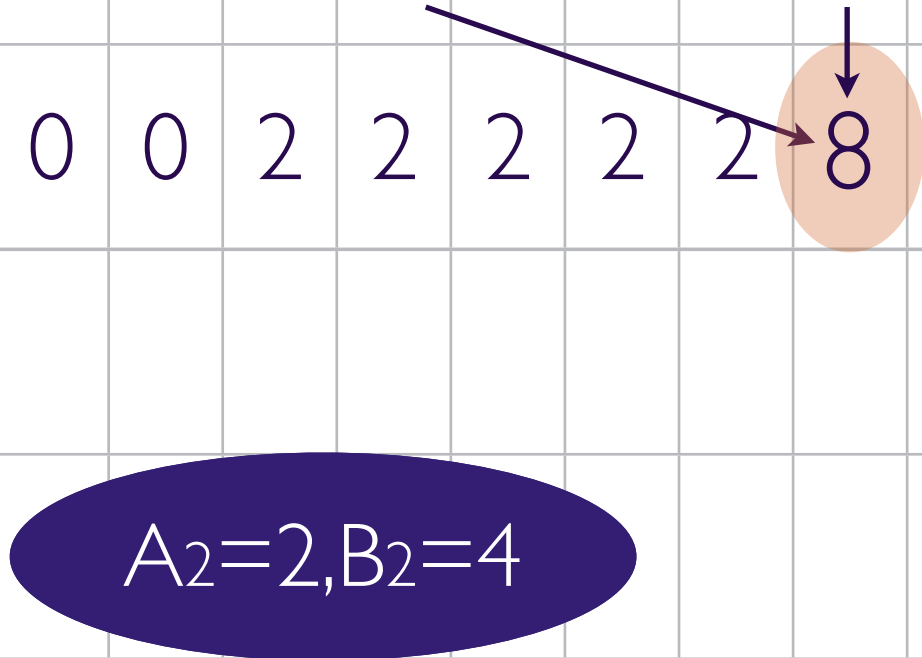
dp	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
1	0	0	0	0	0	0	0	0	0	8	8	8	8	8	8	8	8	8	8	8	8
2	0	0	0	0	2	2	2	2	2												
3																					
4																					
5																					



表を埋める

- $dp(t, i+1) = \text{下の2つのmax}$
 - 夜店 $i+1$ で遊ぶ $\rightarrow$ 時刻 $(t - B_{i+1})$ で夜店 i まで遊んだ時の楽しさ $+ A_{i+1}$
 - 夜店 $i+1$ で遊ばない $\rightarrow$ 時刻 t で夜店 i まで遊んだ時の楽しさ
- 数式で書くと $dp(t, i+1) = \max\{dp(t - B_{i+1}, i) + A_{i+1}, dp(t, i)\}$

dp	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
1	0	0	0	0	0	0	0	0	0	8	8	8	8	8	8	8	8	8	8	8	8
2	0	0	0	0	2	2	2	2	2	8											
3																					
4																					
5																					



表を埋める

- $dp(t, i+1) = \text{下の2つのmax}$
 - 夜店 $i+1$ で遊ぶ $\rightarrow$ 時刻 $(t - B_{i+1})$ で夜店 i まで遊んだ時の楽しさ $+ A_{i+1}$
 - 夜店 $i+1$ で遊ばない $\rightarrow$ 時刻 t で夜店 i まで遊んだ時の楽しさ
- 数式で書くと $dp(t, i+1) = \max\{dp(t - B_{i+1}, i) + A_{i+1}, dp(t, i)\}$

dp	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
1	0	0	0	0	0	0	0	0	0	8	8	8	8	8	8	8	8	8	8	8	8
2	0	0	0	0	2	2	2	2	2	8	8	8	8	10	10	10	10	10	10	10	10
3	0	0	0	0	2	2	2	2	2	8	8	8	8	10	10	10	10	10	10	10	10
4	0	0	0	6	6	6	6	8	8	8	8	8	14	14	14	14	16	16	16	16	16
5	0	0	0	6	6	6	6	8	8	8	8	11	14	14	14	14	16	16	16	16	19

計算量

- 表のマス目は $N \times T = 3000 \times 3000 = 9000000$ (10^7 くらい)個
- できそう！
- $O(NT)$
- 参考:10点の解法では $2^{3000} = 100\ldots\ldots00$ (10^{900} くらい) の組み合わせを考えていた

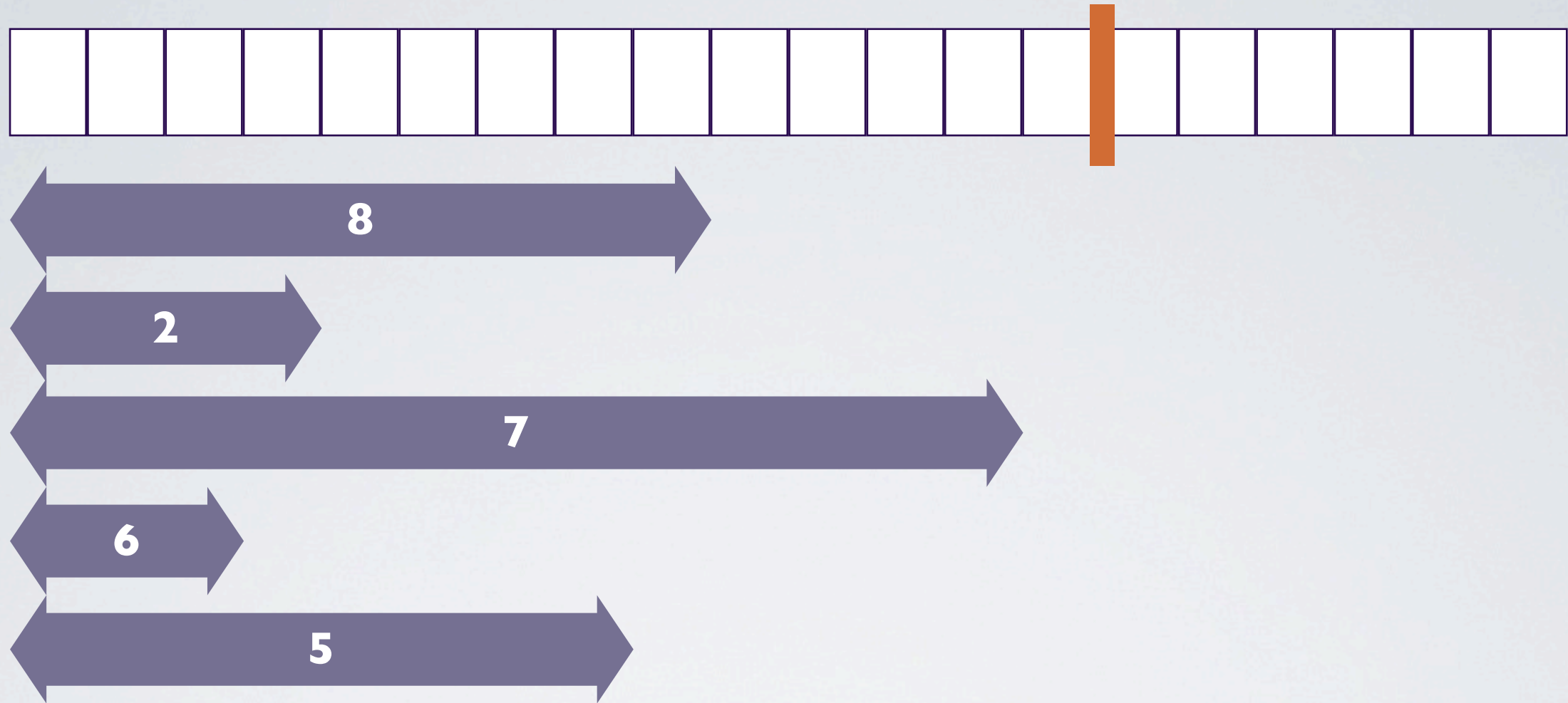
計算量

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
1	0	0	0	0	0	0	0	0	0	8	8	8	8	8	8	8	8	8	8	8	8
2	0	0	0	0	2	2	2	2	2	8	8	8	8	10	10	10	10	10	10	10	10
3	0	0	0	0	2	2	2	2	2	8	8	8	8	10	10	10	10	10	10	10	10
4	0	0	0	6	6	6	6	8	8	8	8	8	14	14	14	14	16	16	16	16	16
5	0	0	0	6	6	6	6	8	8	8	8	11	14	14	14	14	16	16	16	16	19

- 表のマス目は $N \times T = 3000 \times 3000 = 9000000$ (10^7 くらい)個
- できそう！
- $O(NT)$
- 参考:10点の解法では $2^{3000} = 100\ldots\ldots 00$ (10^{900} くらい) の組み合わせを考えていた

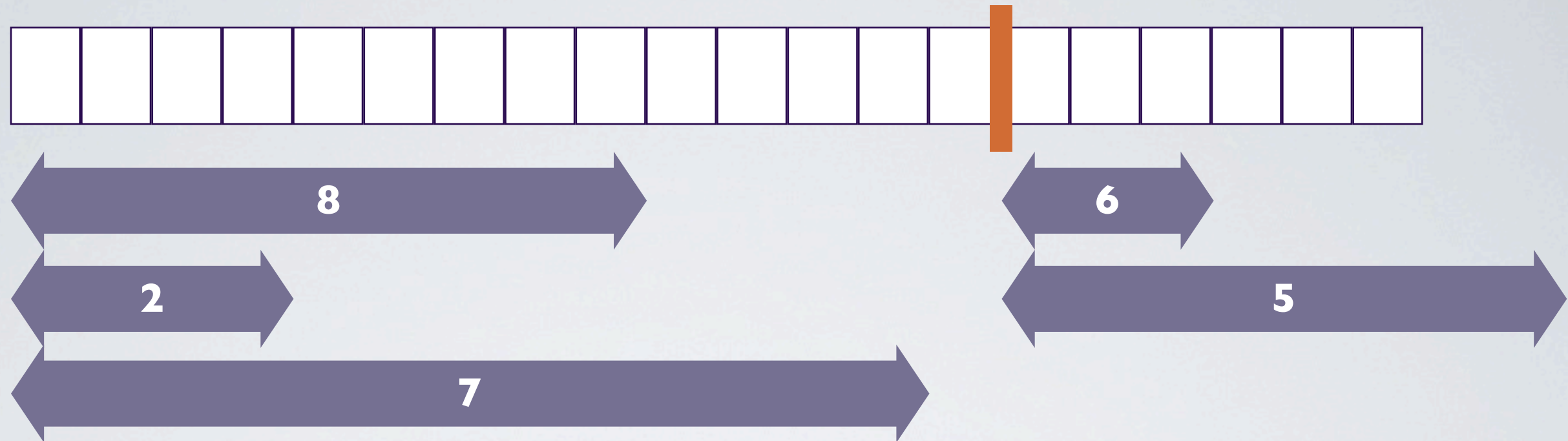
ここまでで30点

じゃあ花火が途中であったらどうするよ



- 花火までに行く夜店 と 花火の後に行く夜店 の2つにとりあえず分けてみる
- 分け方は $N+1$ =約3000通りしかないから大丈夫じゃない？
- →ナップサック問題を2つ解けばいい！

じゃあ花火が途中であったらどうするよ



- 花火までに行く夜店 と 花火の後に行く夜店 の2つにとりあえず分けてみる
- 分け方は $N+1$ =約3000通りしかないから大丈夫じゃない？
- →ナップサック問題を2つ解けばいい！

できそうだけど...？

- 表のマス目が $N \times T = 9000000$ 個
- 場合分けが3000通り
- 掛け算すると270000000000(10^{10} 以上)
- 1秒に収めるにはちょっと厳しい.
- $O(N^2T)$

表を眺めてみる

dp	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
1	0	0	0	0	0	0	0	0	0	8	8	8	8	8	8
2	0	0	0	0	2	2	2	2	2	8	8	8	8	10	10
3	0	0	0	0	2	2	2	2	2	8	8	8	8	10	10
4	0	0	0	6	6	6	6	8	8	8	8	8	14	14	14
5	0	0	0	6	6	6	6	8	8	8	8	11	14	14	14

- 花火があるまでに， 夜店1まで遊ぶ， 夜店2まで遊ぶ,..., 夜店Nまで遊ぶ， は全部同じ表で求まる！
- 花火の後の部分も同様.
- ただし $dp(t,i)$ =時刻tから終了時刻までに夜店i~Nを遊ぶ， として考えなければならない

完成！

dp1	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14		14	15	16	17	18	19	20	dp2
1	0	0	0	0	0	0	0	0	0	8	8	8	8	8	8	0	6	6	6	6	0	0	0	1
2	0	0	0	0	2	2	2	2	2	8	8	8	8	10	10	0	6	6	6	6	0	0	0	2
3	0	0	0	0	2	2	2	2	2	8	8	8	8	10	10	0	6	6	6	6	0	0	0	3
4	0	0	0	6	6	6	6	8	8	8	8	8	14	14	14	0	6	6	6	6	0	0	0	4
5	0	0	0	6	6	6	6	8	8	8	8	11	14	14	14	0	0	0	0	0	0	0	0	5

- $dp1(S,i) + dp2(S,i+1)$ の最大値がMの最大値！

(この場合は16)

- $N \times T$ 個のマスを埋める + N 通りの組み合わせを見る
時間的にも大丈夫そう $O(NT)$

ちなみに

解法はひとつ!じゃない!!

- 表を左から右に, 上から下に見ていく
- tt =時刻 t 以降に夜店 $i+1$ で遊びはじめて遊び終わる時刻とする
 $tt = \text{if}(t < S < t + B_{i+1}) \text{ then } (S + B_{i+1}) \text{ else } (t + B_{i+1})$
- $dp(t, i) + A_{i+1}$ を $dp(tt, i+1)$ と比べて大きければ書き込む
- $dp(t, i)$ を $dp(t+1, i)$ と比べて大きければ書き込む
- $dp(t, i)$ を $dp(t, i+1)$ と比べて大きければ書き込む

dp	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
1	0	0	0	0	0	0	0	0	0	8	8	8	8	8	8	8	8	8	8	8	8
2	0	0	0	0	2	2	2	2	2	8	8	8	8	10	10	10	10	10	10	10	10
3	0	0	0	0	2	2	2	2	2	8	8	8	8	10	10	10	10	10	10	10	10
4	0	0	0	6	6	6	6	8	8	8	8	8	14	14	14	14	16	16	16	16	16
5	0	0	0	6	6	6	6	8	8	8	8	11	14	14	14	14	16	16	16	16	16

ちなみ2

DPについてもっと知りたい!

- プログラミングコンテストでの動的計画法

<http://www.slideshare.net/iwiwi/ss-3578511>



- 最強最速アルゴリズムマー養成講座

<http://www.itmedia.co.jp/enterprise/articles/1003/06/news002.html>

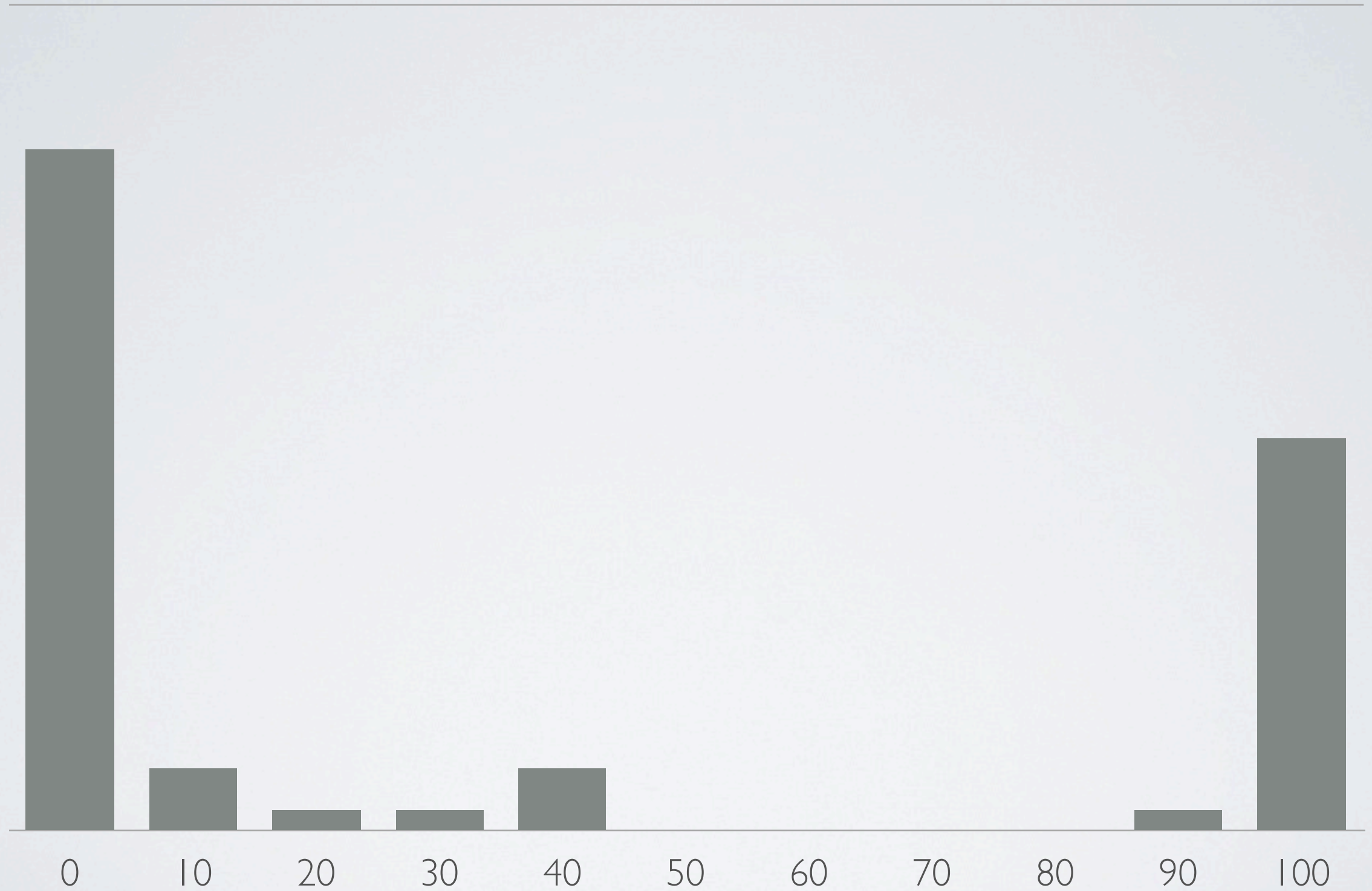
<http://www.itmedia.co.jp/enterprise/articles/1005/15/news002.html>



- どちらもタイトルでgoogle検索すればすぐ出ます

ちなみ3

結果





ありがとうございました

解説: 釘 (NAILS)

第11回 日本情報オリンピック本選

今城 健太郎 (京都大学, IOI 2006 メキシコ大会)

問題の概要

正三角形で囲われた釘の数を求める問題

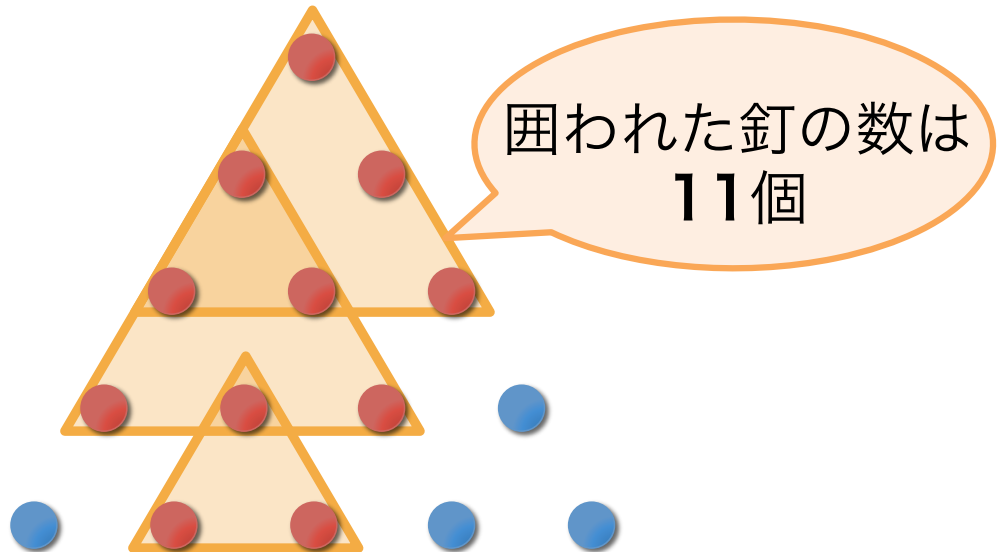
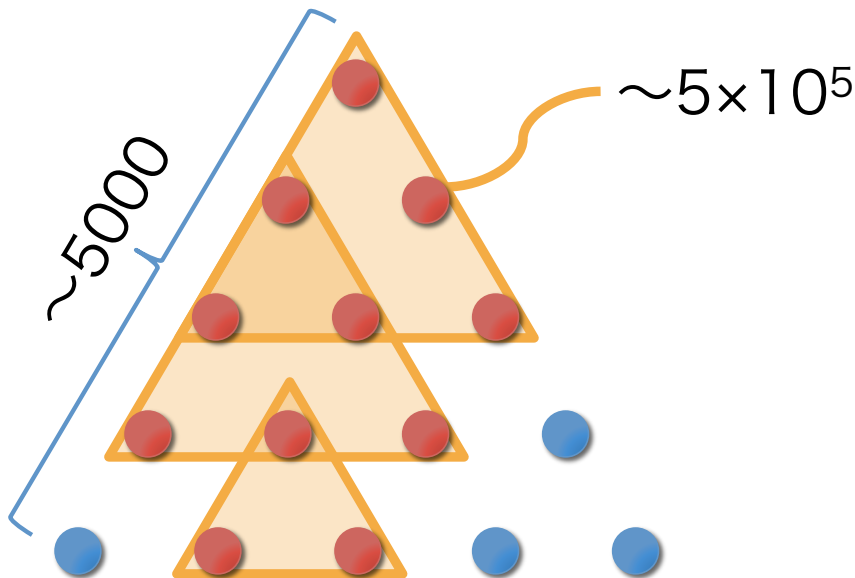


図: 大きさ5の正三角形

問題の制約

正三角形の一辺の大きさ **5000以下**
囲む正三角形の数 **5×10^5 以下**

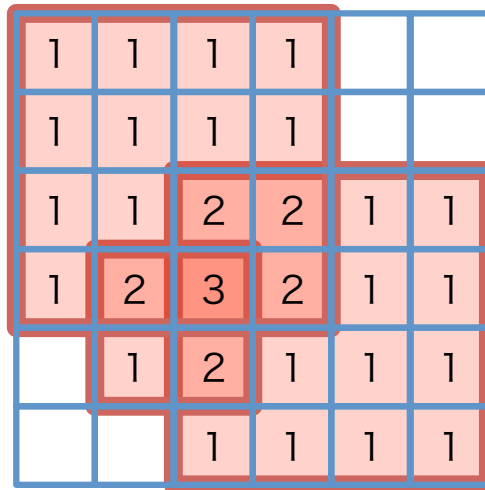


解法

- 累積和による解法
- 最大値を伝搬する解法

四角形の累積和問題

- 四角形の領域に対して加減算を行う
- 2008年本選5番などで出題
- 素朴な計算方法だと $O(\text{数} \times \text{面積})$



四角形の累積和問題

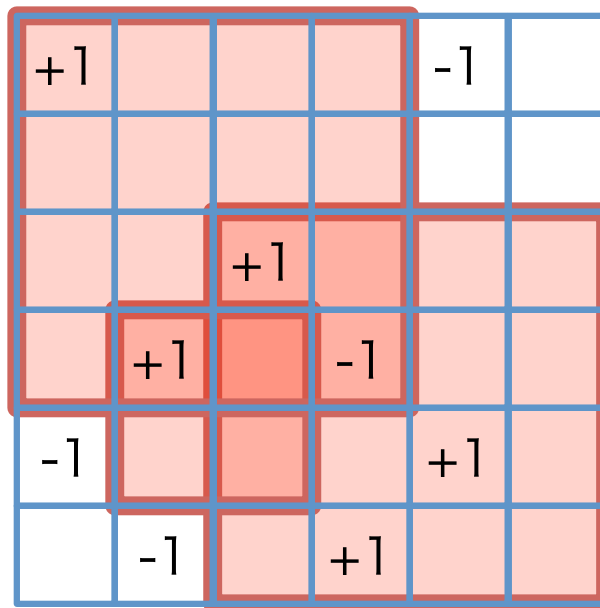
塗りつぶす計算を横着し計算量を

$O(\text{数} \times \text{面積}) \rightarrow O(\text{数} + \text{面積})$ に短縮

- 手順1 すべての四角形について
 各頂点の右下に数字を足す
- 手順2 左から右へ累積和を取る
- 手順3 上から下へ累積和を取る

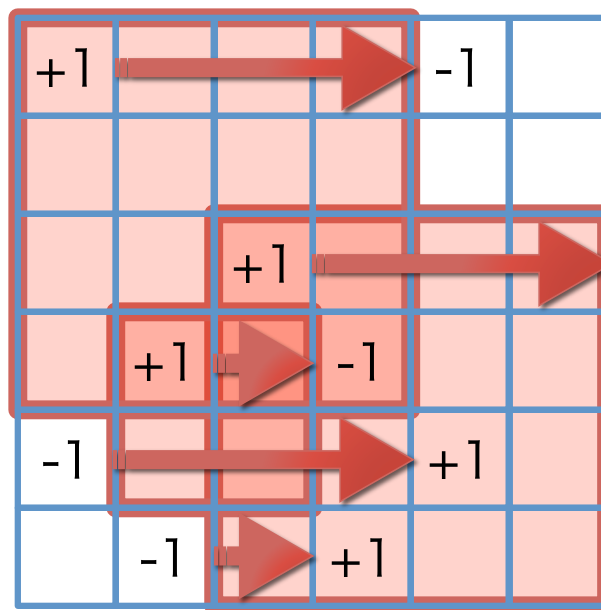
四角形の累積和問題

手順1 すべての四角形について
各頂点の右下に数字を足す



四角形の累積和問題

手順2 左から右へ累積和を取る



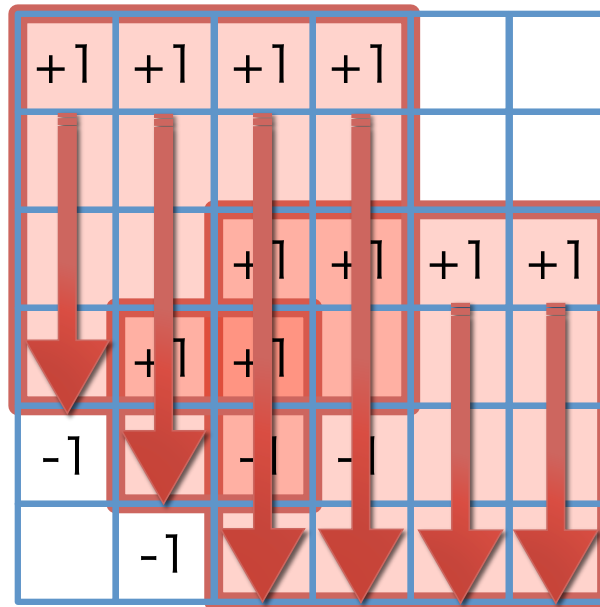
四角形の累積和問題

手順2 左から右へ累積和を取る

+1	+1	+1	+1		
		+1	+1	+1	+1
	+1	+1			
-1	-1	-1	-1		
	-1	-1			

四角形の累積和問題

手順3 上から下へ累積和を取る



四角形の累積和問題

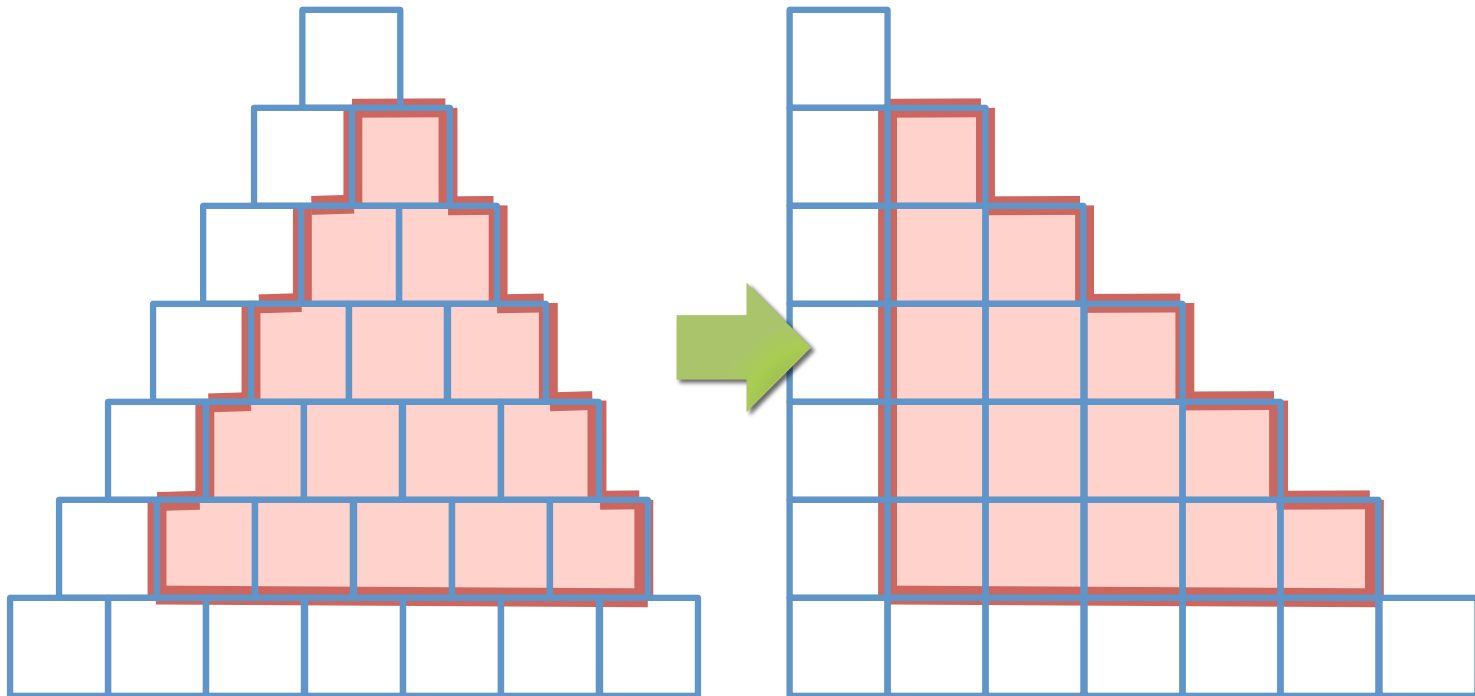
手順3 上から下へ累積和を取る

完成！

1	1	1	1		
1	1	1	1		
1	1	2	2	1	1
1	2	3	2	1	1
	1	2	1	1	1
		1	1	1	1

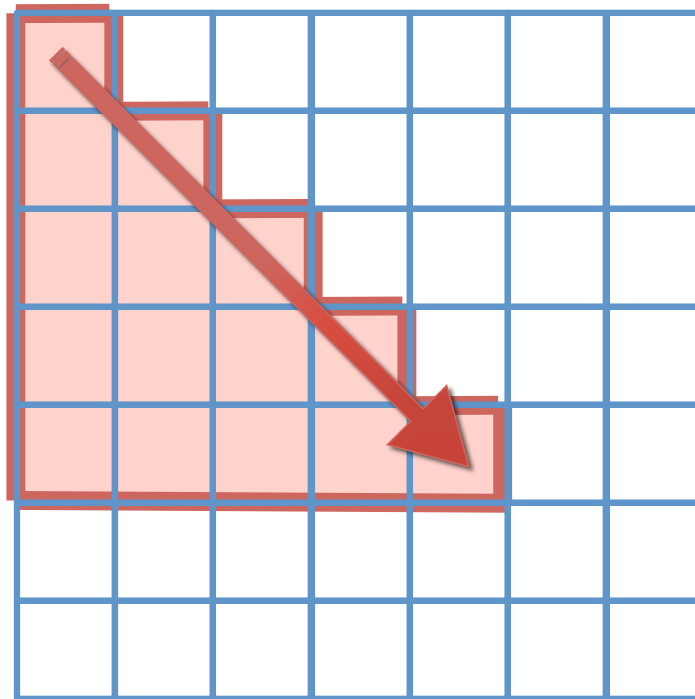
正三角形の累積和問題

斜めありの四角形問題に再解釈できる



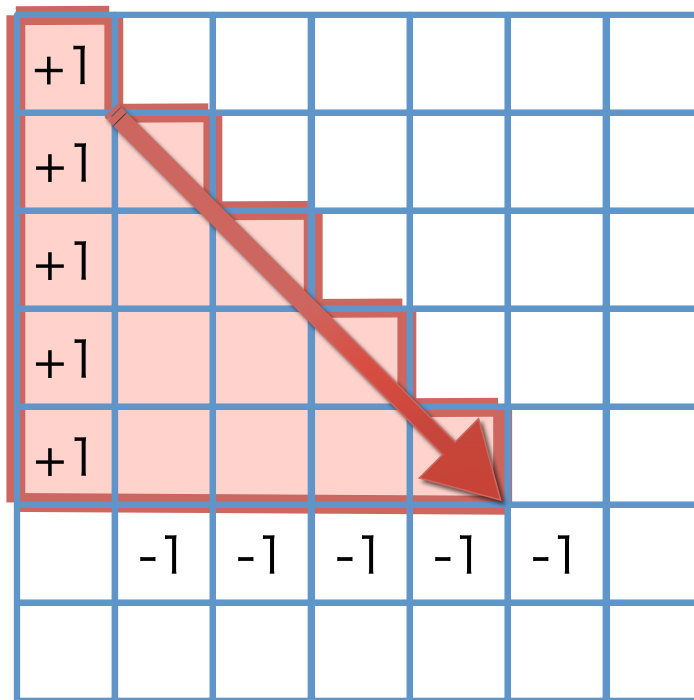
正三角形の累積和問題

斜めありの四角形は斜めに注目



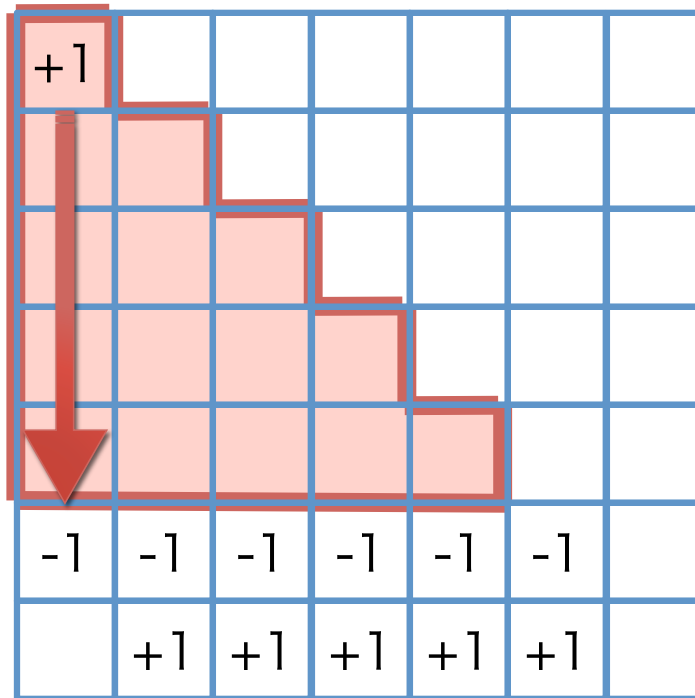
正三角形の累積和問題

最後の累積和を左上から右下とすると



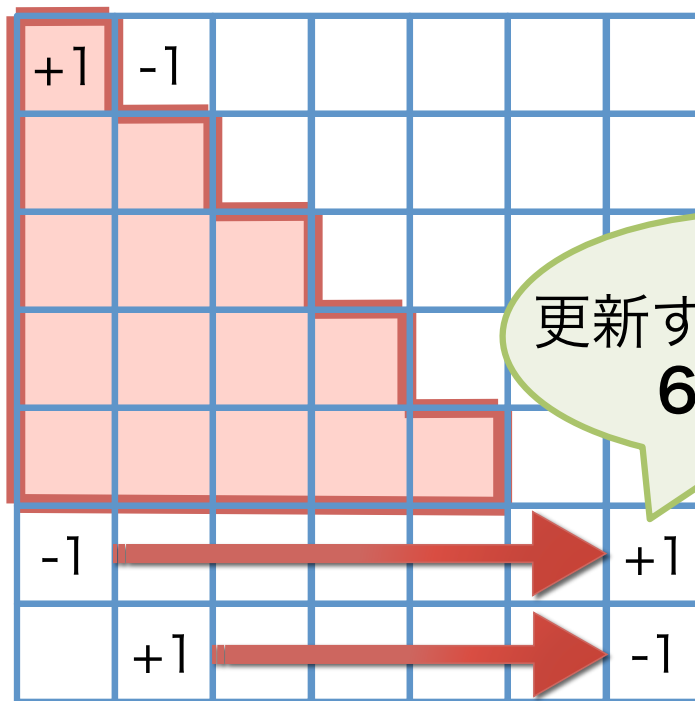
正三角形の累積和問題

その前の累積和は上から下なので



正三角形の累積和問題

その前の累積和は左から右なので



更新すべき場所は
6個のみ

正三角形の累積和問題

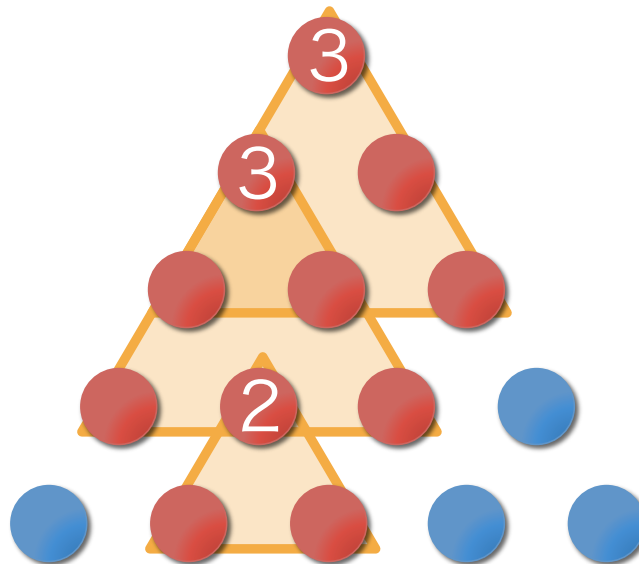
塗りつぶす計算を横着し計算量を

$O(\text{数} \times \text{面積}) \rightarrow O(\text{数} + \text{面積})$ に短縮

- 手順1 適切な箇所に数字を足す
- 手順2 左から右へ累積和を取る
- 手順3 上から下へ累積和を取る
- 手順4 左上から右下へ累積和を取る

最大値の伝搬

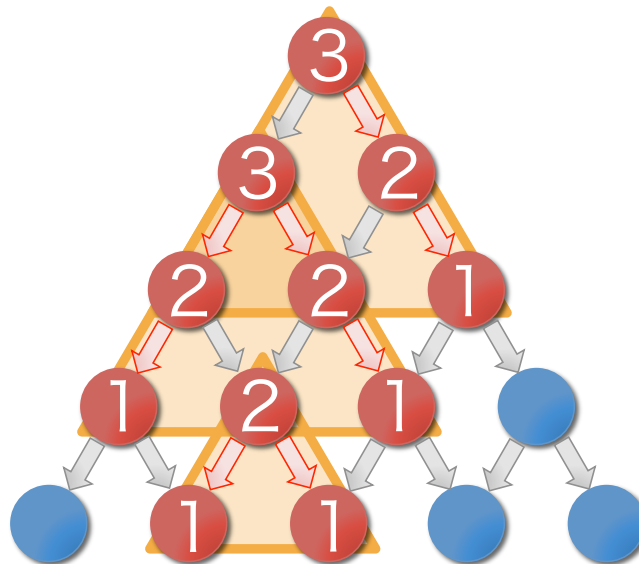
一番上の頂点に大きさを入れる



最大値の伝搬

上から順に以下の式を適用する

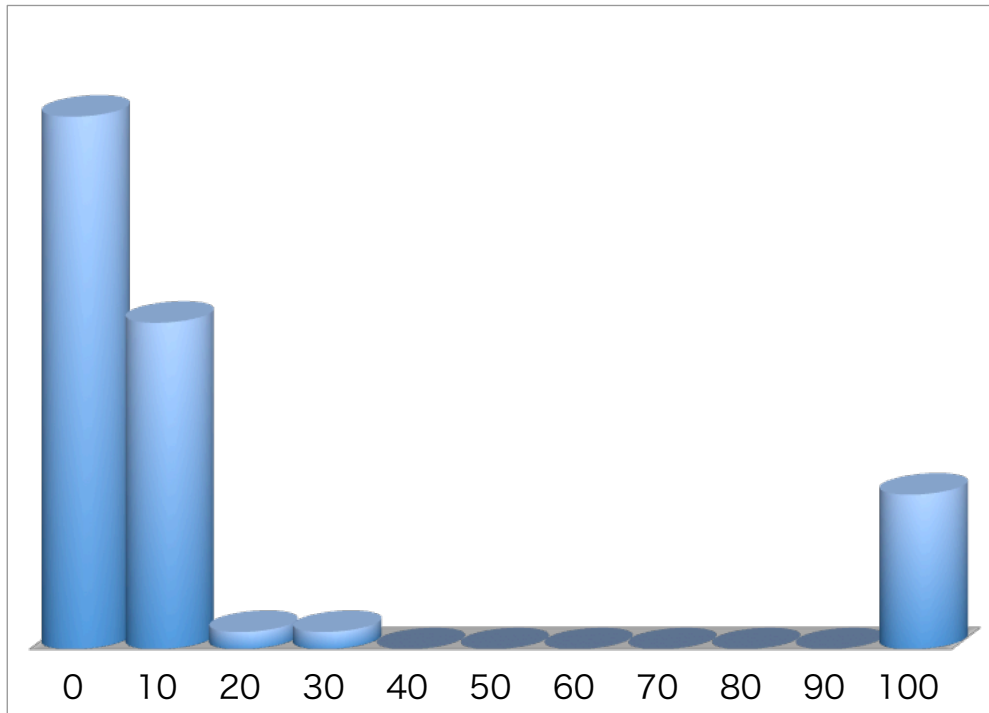
自分 = $\max(\text{自分}, \max(\text{左上}, \text{右上}) - 1)$

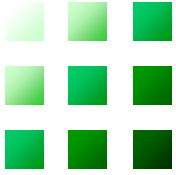


解法の概要

- 1a. 正三角形の累積和問題を解く
 - 1) 各正三角形について
適切な箇所(計6ヶ所)に数字を足す
 - 2) 左から右へ累積和を取る
 - 3) 上から下へ累積和を取る
 - 4) 左上から右下へ累積和を取る
- 1b. 最大値の伝搬を行う
- 2. 0でないマス数を数える

得点分布





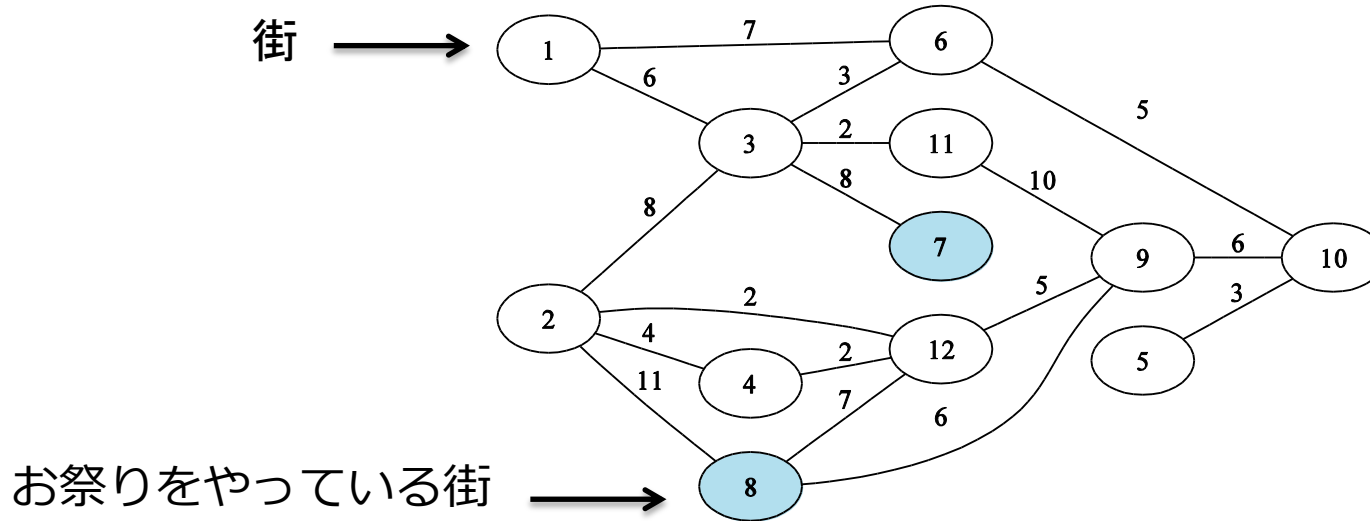
第 11 回 日本情報オリンピック本選 問題5 JOI 国のお祭り事情

秋葉 拓哉

(東京大学 情報理工学系研究科)



問題概要



- できるだけお祭りに近づきたくない
- 街 s_i から街 t_i にどれだけ距離を保って移動できるか？
- ...という質問に大量に答えたい (Q 個)



用語など

グラフ

- 街 = 頂点
- 道 = 辺, 枝

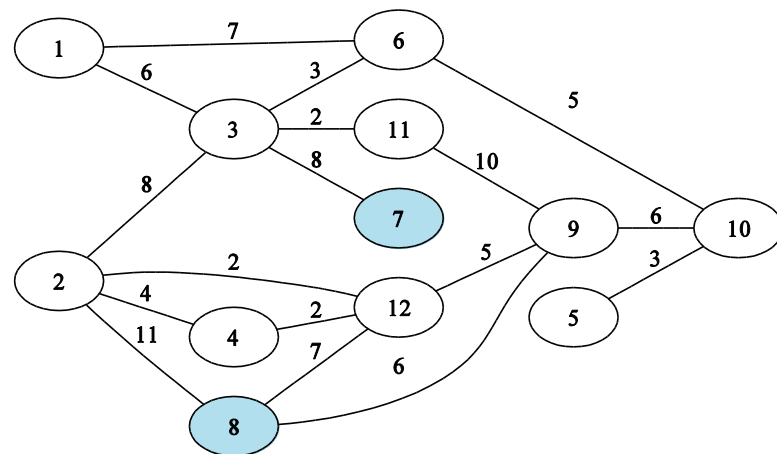
変数

N : 街, M : 道, Q : クエリ

スコア

この解説では,
各街の, お祭りまでの距離を, スコアと呼びます

できるだけスコアの小さな町を通らず移動したい





$O(Q M \log N)$ の解法 (10点)



10 点解法：ステップ 1

- まず、各街についてスコアを求める
 - スコア = お祭りまでの距離
- お祭りをしている街から **Dijkstra**
 - Dijkstra : グラフで最短経路を求めるアルゴリズム
 - 今日は詳細は省略します（調べてください）
- 計算量は？
 - $O(M \log N)$



10 点解法：ステップ 2

- 各クエリ (s_i, t_i) に対し, 最も良い経路を求める
- また, **Dijkstra** しましょう
 - 辺でなく頂点にコストがついている
 - コストは加算するのではなく最小を取る
- 計算量は？
 - 各クエリ $O(M \log N)$
 - クエリ Q 個に対し $O(QM \log N)$

他にもいくつか解法があります

- Dijkstra を使う方法（今話した）
- Union-Find を使う方法（次に紹介）
- 2 分探索を使う方法（省略）



10 点別解

- 同様に各街についてお祭りまでの距離を求める
- 全道を, スコアの大きい順にソート
- 各クエリ (s_i, t_i) は以下のように処理
 - はじめ, 全部の道は無かったと考える
 - 道を順に加えてゆく
 - 街 s_i と街 t_i が繋がったらやめる
 - 最後に加えた道のスコアが答え



10 点別解

- 道を加えていき、繋がっているかを判定したい
- 道を加える度に深さ優先探索で判定？
 - 毎回 $O(M)$ 時間かかってしまい非効率的
- Union-Find のデータ構造を使うと効率的
 - 毎回 $O(\alpha(N))$ 時間
 - 説明しません（ごめんね）
- 計算量： $O(M \log N + Q M \alpha(N))$

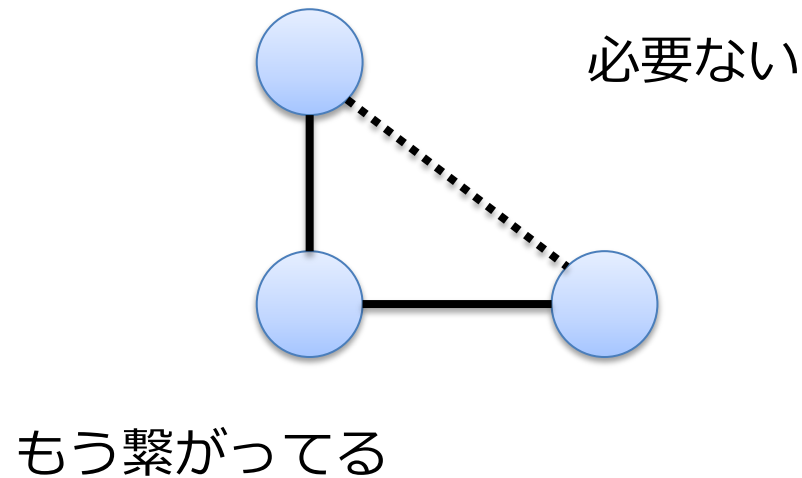


**$O(M \log N + Q N \alpha(N))$ の解法
(30点)**



30 点解法

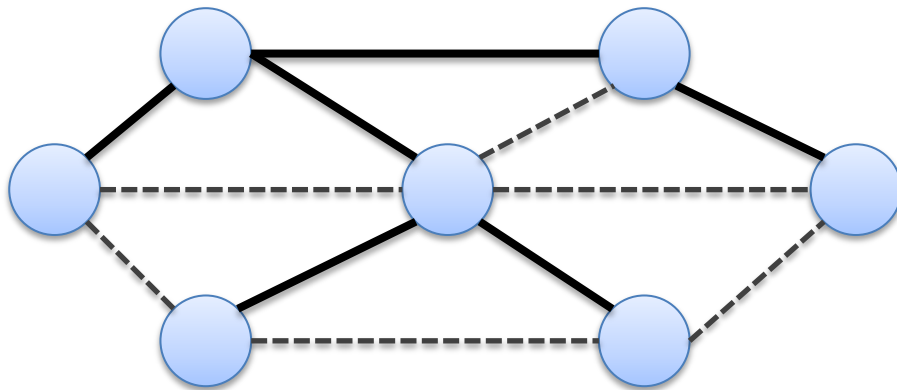
- つなぐ 10 点解法をよく考えてみよう
- 既に繋がっているところを道でつなぐ意味なし
 - 特に新しいところに行けるようにならない
 - いらない





30 点解法

- どの道が有用かをはじめに調べる
 - $O(M \alpha(N))$ でできる (Union-Find)
 - $N - 1$ 本の道が有用として残る (全域木になるので)
- 各クエリに対し N 本の辺だけ見ればよい！
 - $O(QM \alpha(N)) \rightarrow O(QN \alpha(N))$



点線の辺は無かったことに
↓
道が M 本から $N - 1$ 本に！

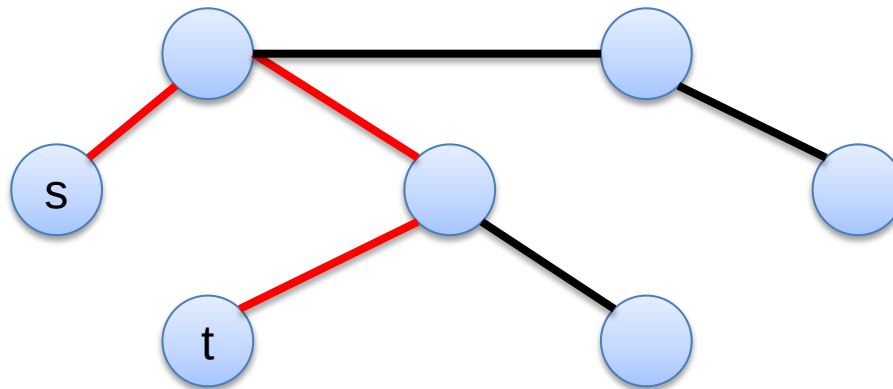


$O((M + Q) \log N)$ の解法 (100 点)



100 点解法

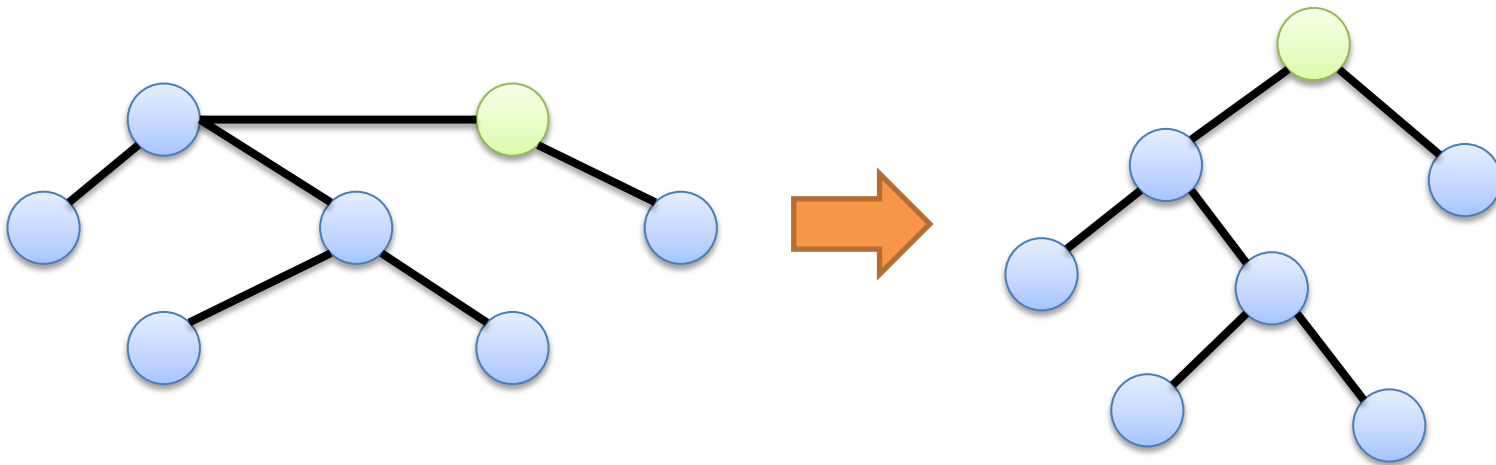
- つなぐ解法をさらによく考える
- 経路は, つないだ全域木の上での経路になる
- 木の中の経路に含まれる全ての街の中で, スコアの最小値が答え





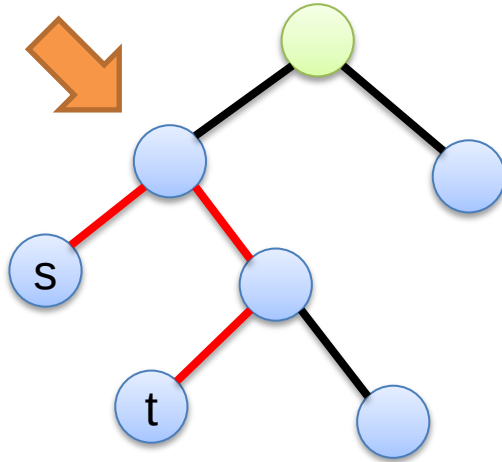
100 点解法

- 適当な頂点を根にして，根付き木にしよう





Lowest Common Ancestor



経路を 2 つに分けて考える

- $s \rightarrow \text{LCA}$
- $t \rightarrow \text{LCA}$

- Lowest Common Ancestor とその応用
 - LCA と, さらにそこまでの経路のスコアの最大値
 - ダブリング等を用いれば良い (省略)
 - 前計算 $O(N \log N)$, クエリ $O(\log N)$



100 点解法

- 少し工夫すると LCA だけで OK
 - 木を, 頂点が上に行くほどスコアが低くなるようにうまく作る
 - LCA のスコアがそのまま答えに

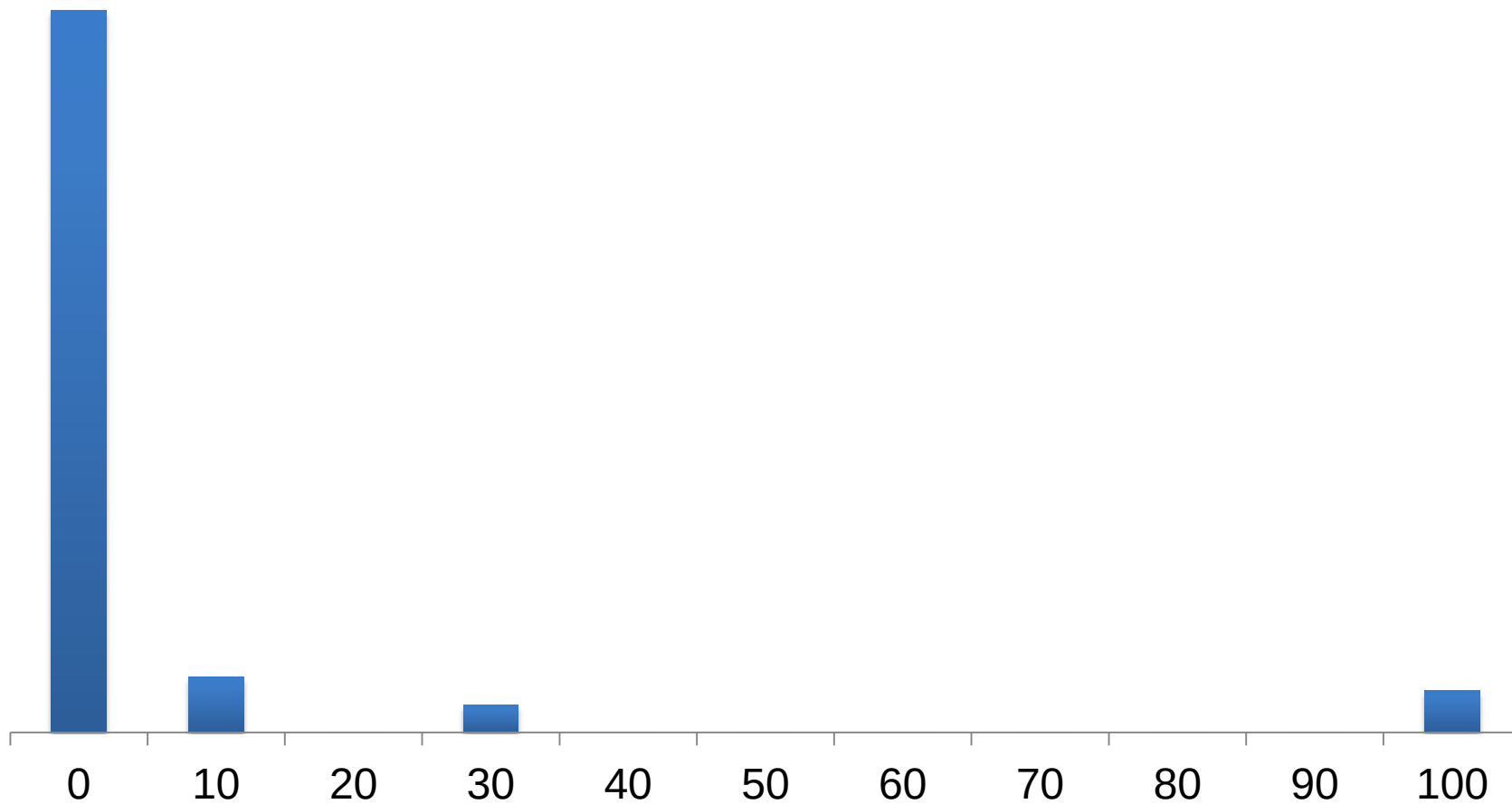


別解など

- クエリを頂点に集合として貯めこみ，マージしてゆく
 - マージ時にぶつかったら，そこがそのクエリの答え
 - $O(Q \log^2 N)$ になってしまうが多分間に合う，実装楽
- Union-Find を永続データ構造的に実装
 - 実際には縮約をなくせば親へのリンクに時間がつくだけ
 - つながる瞬間を調べたいので，時間で二分探索 $\rightarrow O(Q \log^2 N)$
 - (二分探索やめて LCA すると $O(\log \log n)$ とかになってオモロイ)
- (おまけ：計算量だけなら)
 - 最短路は $O(M)$ のアルゴリズムあり
 - LCA は $O(N), O(1)$ のアルゴリズムあり
 - あわせて， $O(M + Q)$ で解ける，入力の線形 = 最適！ (実際は遅い)



点数分布





最後に

- 多くの重要なキーワードが登場しました
 - Dijkstra のアルゴリズム
 - 深さ優先探索
 - 二分探索
 - Union-Find のデータ構造
 - Lowest Common Ancestor
- 基礎的なものから発展的なものまで
 - 知ってれば解ける訳ではない
 - しかし, 知らないと厳しい
 - ぜひ復習しておいてください